



Accelerating Inner-loop Developer Workflow and Productivity using Docker

21 July 2026



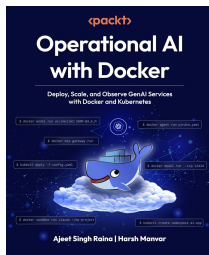
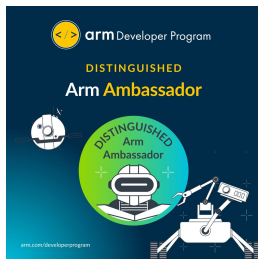
Whale, Hello There 🐳

- Developer Advocate, Docker
- Lead 20K Bengaluru Meetup community
- Distinguished Arm Ambassador
- Authored “Operational AI with Docker”



Ajeet Raina

<https://ajeetraina.com>



Day 1 (July 21)

Improve your inner-loop development workflow and productivity using Docker

- Inner-loop Development Workflow
- What is a container
- Docker across the SDLC
 - Develop (Docker Desktop)
 - Test (Testcontainers)
 - Build (Docker Build Cloud)
 - Secure (Docker Scout)
 - Deploy (Docker Offload)
- Docker and AI Governance - Overview
- Q&A

Day 2 (July 28)

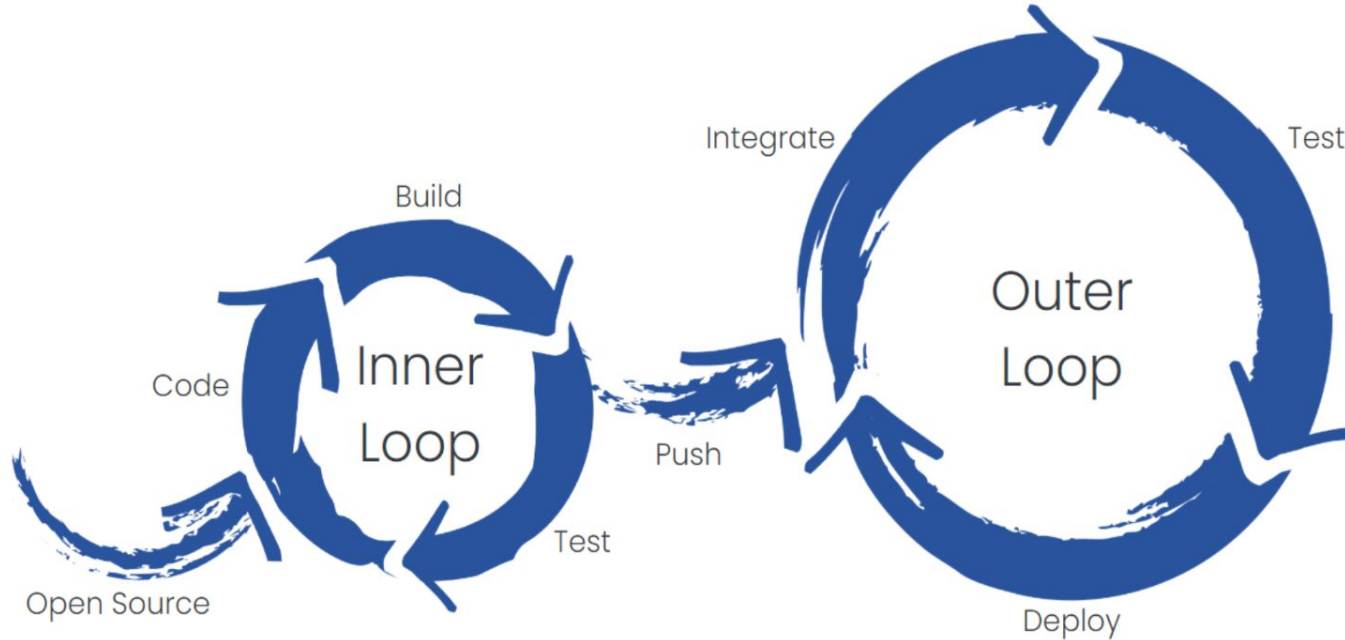
Secure Your Production Environment using Docker

- Container Security Best Practices
- Software Supply Chain and Challenges
- Docker Scout CLI and CI Integration
- Docker Hardened Images - Attestations, SBOMs, VEX, FIPS and SLSA
- DHI in Kubernetes and Helm charts
- Q & A

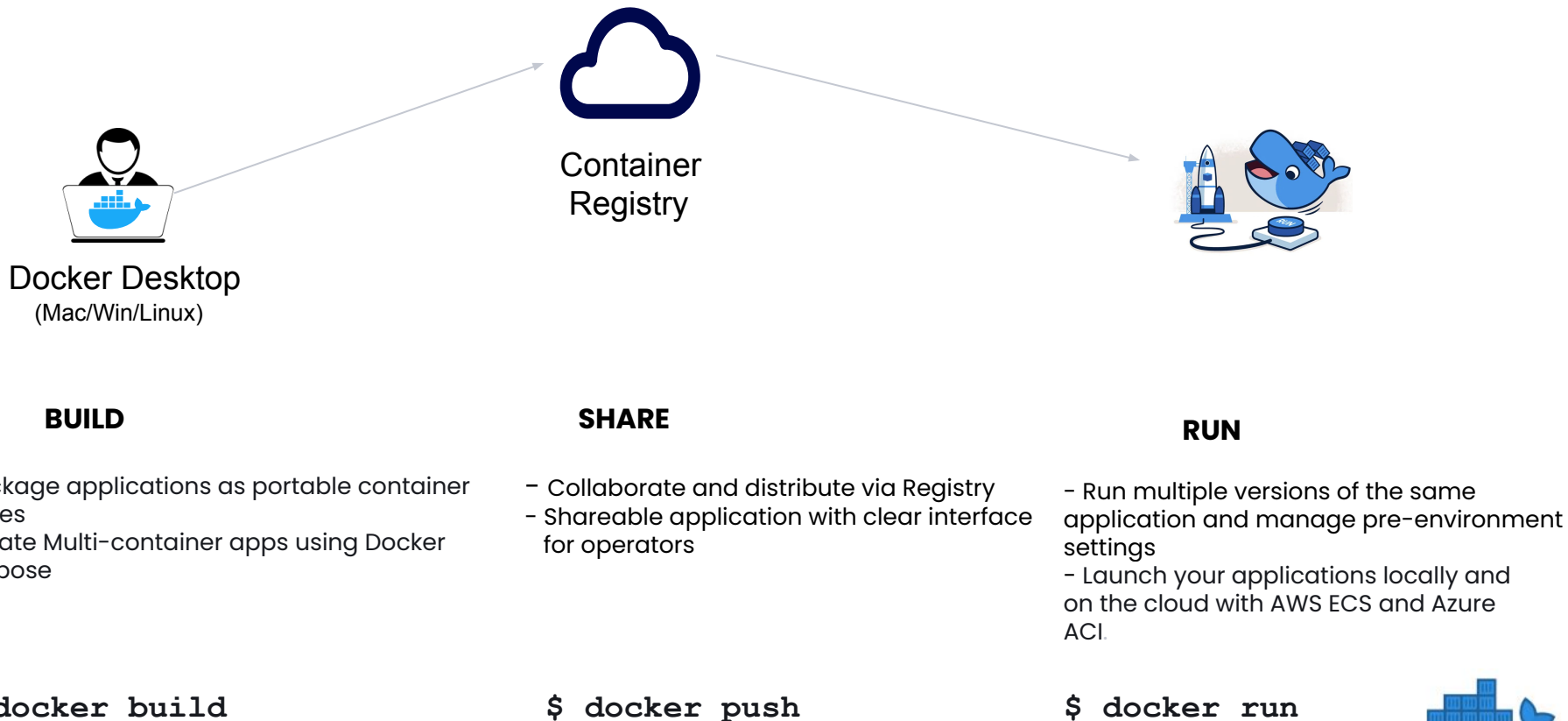


Inner Loop Development Workflow

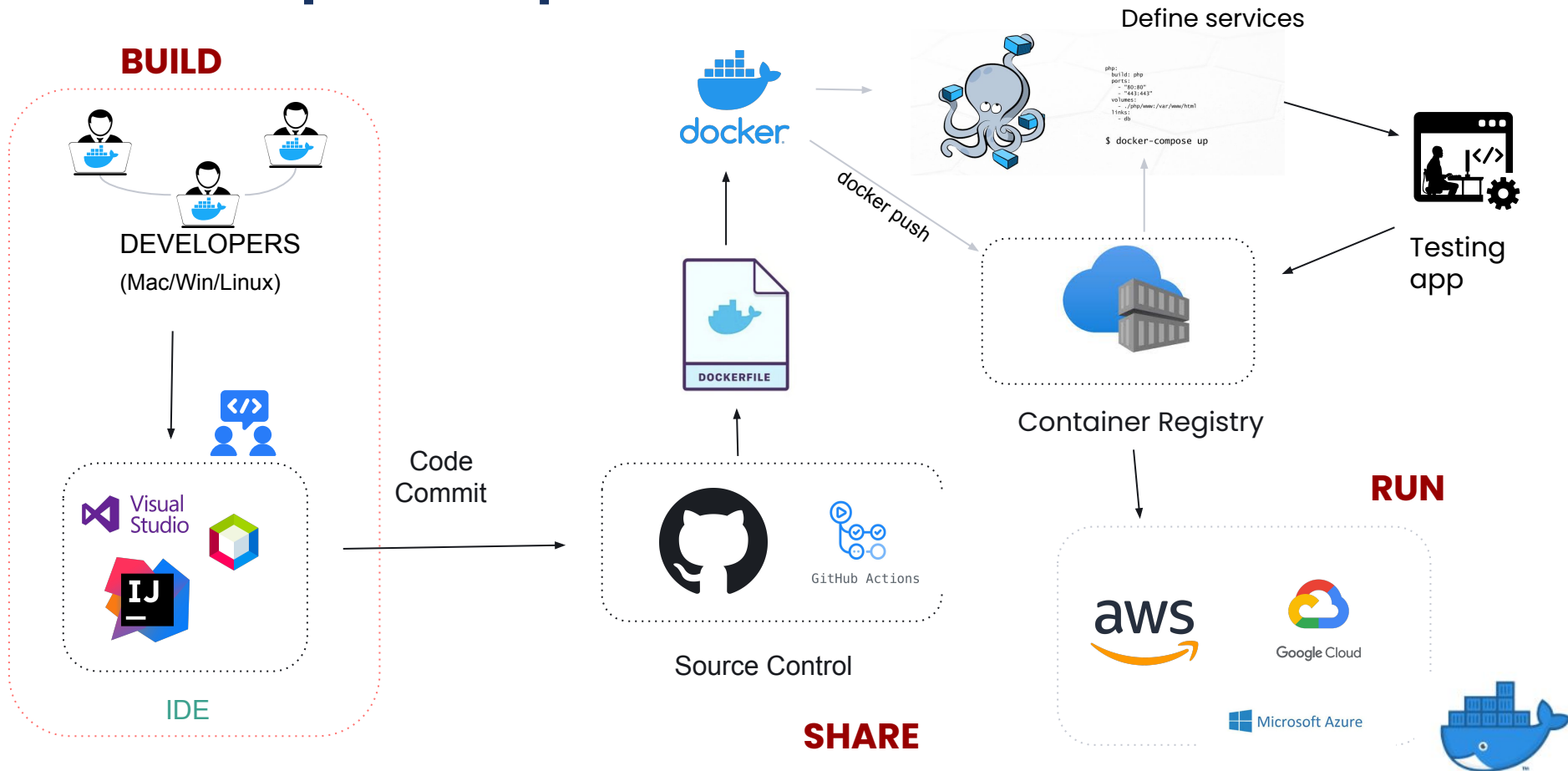
Docker is Uniquely Focused on Developer Success



A 30,000 ft View



Inner-Loop Developer Workflow



Top Developer Productivity Challenges

THE CHALLENGE

-  "Works on my machine" - environments drift across dev, CI, and prod
-  Slow onboarding and painful local setup; long build times
-  Fragmented, context-switch-heavy workflows
-  Developer frustration and burnout driving attrition

HOW DOCKER SOLVES IT

-  Consistent environments → faster, more reliable deployments
-  Simplified setup and accelerated builds
-  Improved, unified development workflows
-  Better developer experience → higher satisfaction & retention

BEFORE DOCKER

Dependency Hell

Manual setup. Conflicting versions. "Works on my machine."



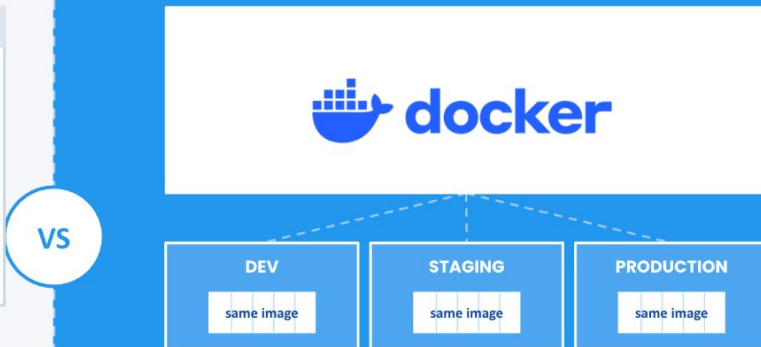
THE PAIN POINTS

- ✗ "It works on my machine"
- ✗ Hours lost to dependency conflicts
- ✗ Onboarding new devs takes days
- ✗ Brittle setup scripts and wikis
- ✗ Inconsistent dev / staging / prod
- ✗ Rollbacks are slow and risky

AFTER DOCKER

Build Once. Run Anywhere.

One image. Identical behavior across every environment.



THE WINS

- ✓ One image — identical everywhere
- ✓ Onboard a new dev in minutes
- ✓ Isolated, reproducible environments
- ✓ Fast, predictable deployments
- ✓ Instant rollbacks (just swap the tag)
- ✓ Same workflow, laptop to cloud
- ✓ Lower infra cost, higher density
- ✓ Foundation for AI / agent workloads

Empowering the Modern Developers



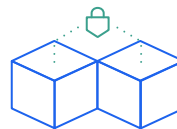
Speed

- Faster Development
- Rapid Prototyping
- Efficient Builds
- Faster Deployment



Security

- Supply Chain Security
- Isolation
- Image Security



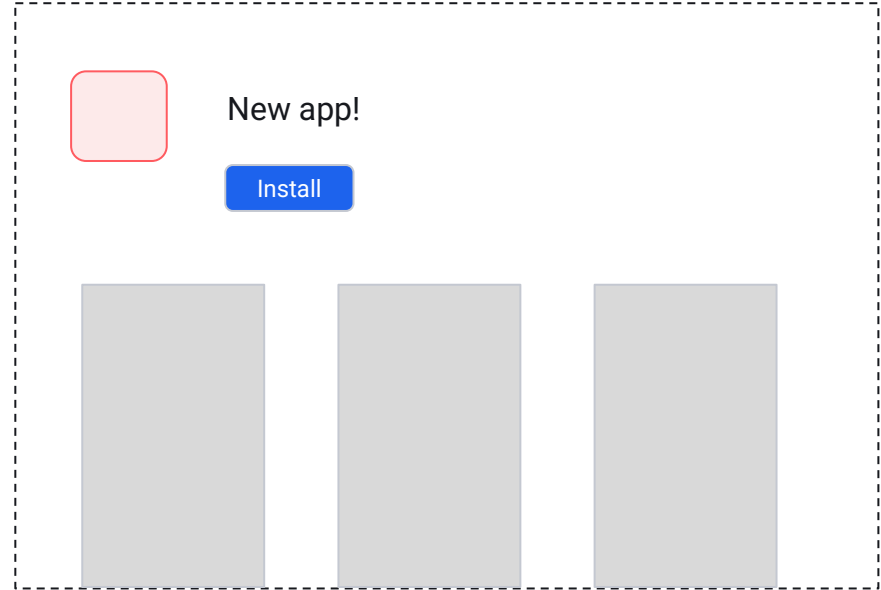
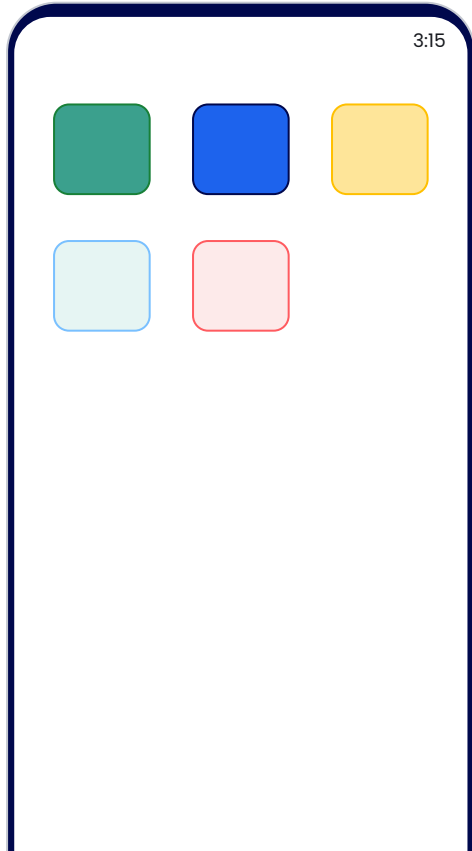
Choice

- Platform Compatibility
- Ecosystem Expansion
- Technology Agnostic

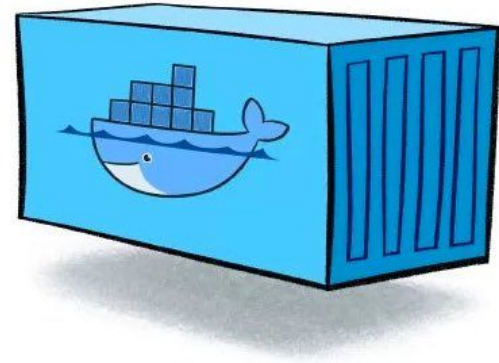


What is a container?

Like smartphone apps...



Images as shipping containers



Container 101

- **Image** = standardized packaging
 - Contains everything needed to do a thing (run an app, provide a runtime, etc.)
 - Images are pushed to/pulled from registries
 - Most often built using a **Dockerfile**
- **Container** = an isolated process running on your machine
 - Uses the image for its filesystem and default configuration
 - Uses kernel namespaces and control groups for isolation
 - Can use mounts to share files between the host and container



Building images

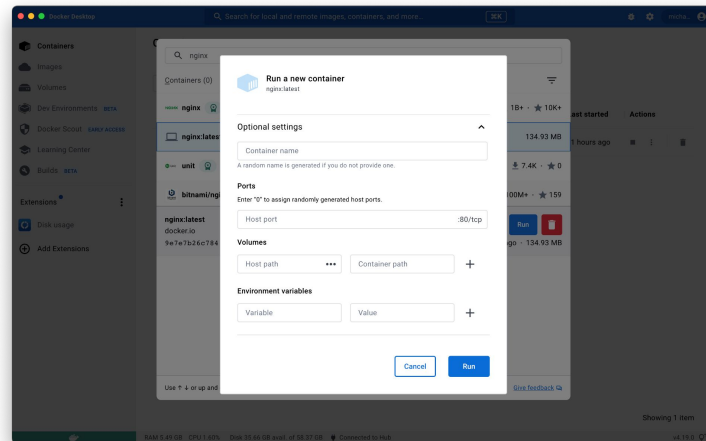
- Use a **Dockerfile**!
- Most often included with source code repositories
- Text-based file with instructions on how to build an image
 - **FROM** – the base image to start from
 - **WORKDIR** – set the working directory in the image
 - **COPY** – copy files from host into the image
 - **RUN** – run a command
 - **CMD** – set the default command
- Build it with **docker build**

```
FROM node:lts
WORKDIR /usr/local/app
COPY package.json yarn.lock ./
RUN yarn install
COPY ./src ./src
CMD ["node", "src/index.js"]
```



Running containers

- Can launch containers using either the CLI or the GUI
 - **docker run** - command to run a container. LOTS of flags available to customize the new container
 - Global search makes it easy to spin up new containers in the GUI



Common CLI flags

- **docker run [OPTIONS] IMAGE [COMMAND]**
 - **-d** - run detached (in the background)
 - **-it** - run in interactive mode with a pseudo TTY
 - **--rm** - automatically remove the container on exit
 - **-p HOST:CONTAINER** - publish a container port on the host
 - **-e KEY=VALUE** - define an environment variable
 - **-v** - mount a volume to the container



Running PostgreSQL, the old way

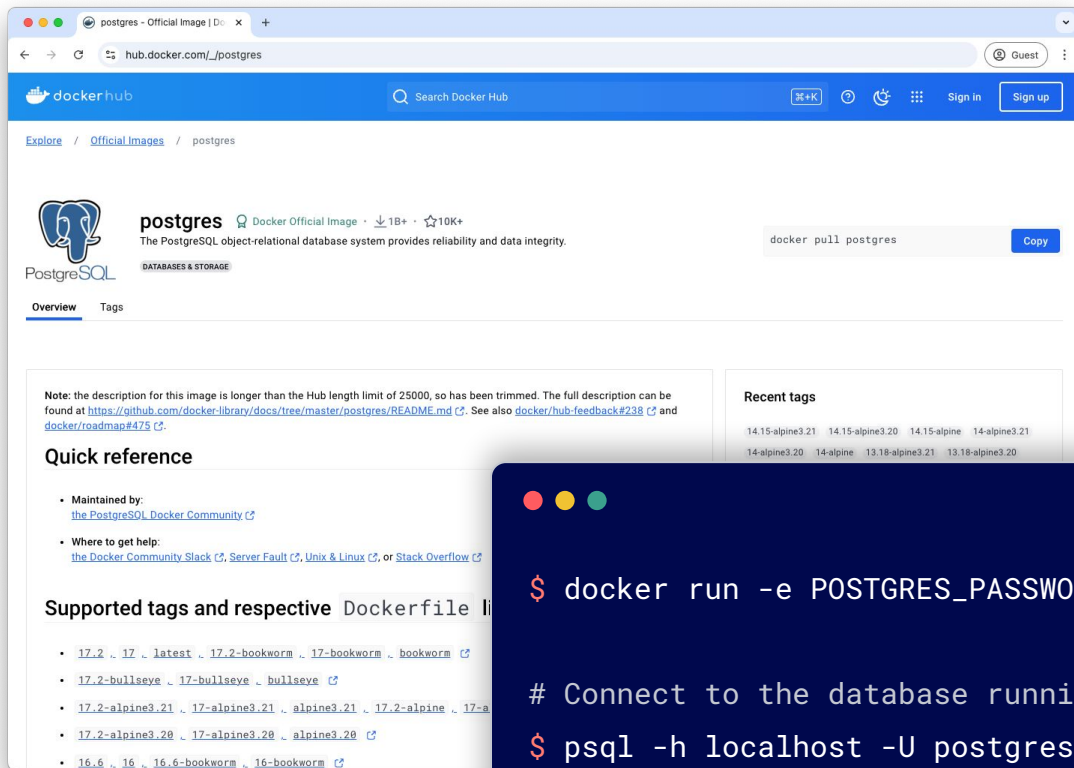
The image displays four overlapping browser windows illustrating the traditional method of obtaining PostgreSQL source code:

- Top Left Window:** The PostgreSQL website's "Downloads" page. It features a navigation bar, a header with the latest release date (November 21, 2024), and a "Quick Links" sidebar. The main content area lists "Downloads" and "Packages and Installation" for various operating systems like Linux and Solaris.
- Top Right Window:** The GitHub repository page for "petere/homebrew-postgresql". It shows the repository name, a search bar, and a list of files and directories. The "About" section mentions it's for Homebrew package manager.
- Bottom Left Window:** The PostgreSQL website's "File Browser" interface. It provides a directory listing of source code files, including various tarballs and documentation for version 17.2.
- Bottom Right Window:** A detailed view of the GitHub repository's file list, showing a table of files and their commit dates. The table includes columns for file names, descriptions, and commit dates.

File	Description	Commit Date
.cirrus.yml	postgresql@16: New	2 years ago
README.md	README: Add note about old versions	4 years ago
formula_renames.json	Migrate to versioned formula scheme	8 years ago
postgresql-common.rb	Add license fields	5 years ago
postgresql@10.rb	PostgreSQL minor releases	3 years ago
postgresql@11.rb	PostgreSQL minor releases	2 years ago
postgresql@12.rb	PostgreSQL minor releases	2 months ago
postgresql@13.rb	PostgreSQL minor releases	2 months ago
postgresql@14.rb	PostgreSQL minor releases	2 months ago
postgresql@15.rb	PostgreSQL minor releases	2 months ago
postgresql@16.rb	PostgreSQL minor releases	2 months ago
postgresql@17.rb	PostgreSQL minor releases	2 months ago
postgresql@8.3.rb	Add livecheck blocks	4 years ago

At the bottom of the image, there is a navigation bar with links for "Policies", "Code of Conduct", "About PostgreSQL", and "Contact". Below this, a copyright notice states: "Copyright © 1996-2025 The PostgreSQL Global Development Group".

Example – Running PostgreSQL



The screenshot shows the Docker Hub page for the 'postgres' official image. The page includes the Docker Hub logo, a search bar, and a 'Sign in' button. The main content area features the 'postgres' logo, a description of the PostgreSQL database system, and a 'docker pull postgres' button. Below this, there is a 'Quick reference' section with links to the PostgreSQL Docker Community, Slack, and other resources. A 'Supported tags and respective Dockerfile links' section lists various tags like '17.2', 'latest', '17.2-bookworm', etc., with corresponding Dockerfile links.

postgres Docker Official Image · 1B+ · 10K+
The PostgreSQL object-relational database system provides reliability and data integrity.

docker pull postgres

Quick reference

- Maintained by: [the PostgreSQL Docker Community](#)
- Where to get help: [the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Supported tags and respective Dockerfile links

- [17.2](#), [17](#), [latest](#), [17.2-bookworm](#), [17-bookworm](#), [bookworm](#)
- [17.2-bullseye](#), [17-bullseye](#), [bullseye](#)
- [17.2-alpine3.21](#), [17-alpine3.21](#), [alpine3.21](#), [17.2-alpine](#), [17-alpine](#)
- [17.2-alpine3.20](#), [17-alpine3.20](#), [alpine3.20](#)
- [16.6](#), [16](#), [16.6-bookworm](#), [16-bookworm](#)



CLI

```
$ docker run -e POSTGRES_PASSWORD=dev -p 5432:5432 postgres:17.2

# Connect to the database running in the container

$ psql -h localhost -U postgres
```



Lab 1

- ✓ Finding images on Docker Hub
- ✓ Ease of downloading and running a containerized service
- ✓ Ability to run multiple versions side-by-side with no conflicts

<https://dockerworkshop.com/lab1/postgres>

Container terminology



Container = Isolated Process

Not a virtual machine. Just a process.
Runs independent of other containers
and what's on the host machine



Image = Standard packaging

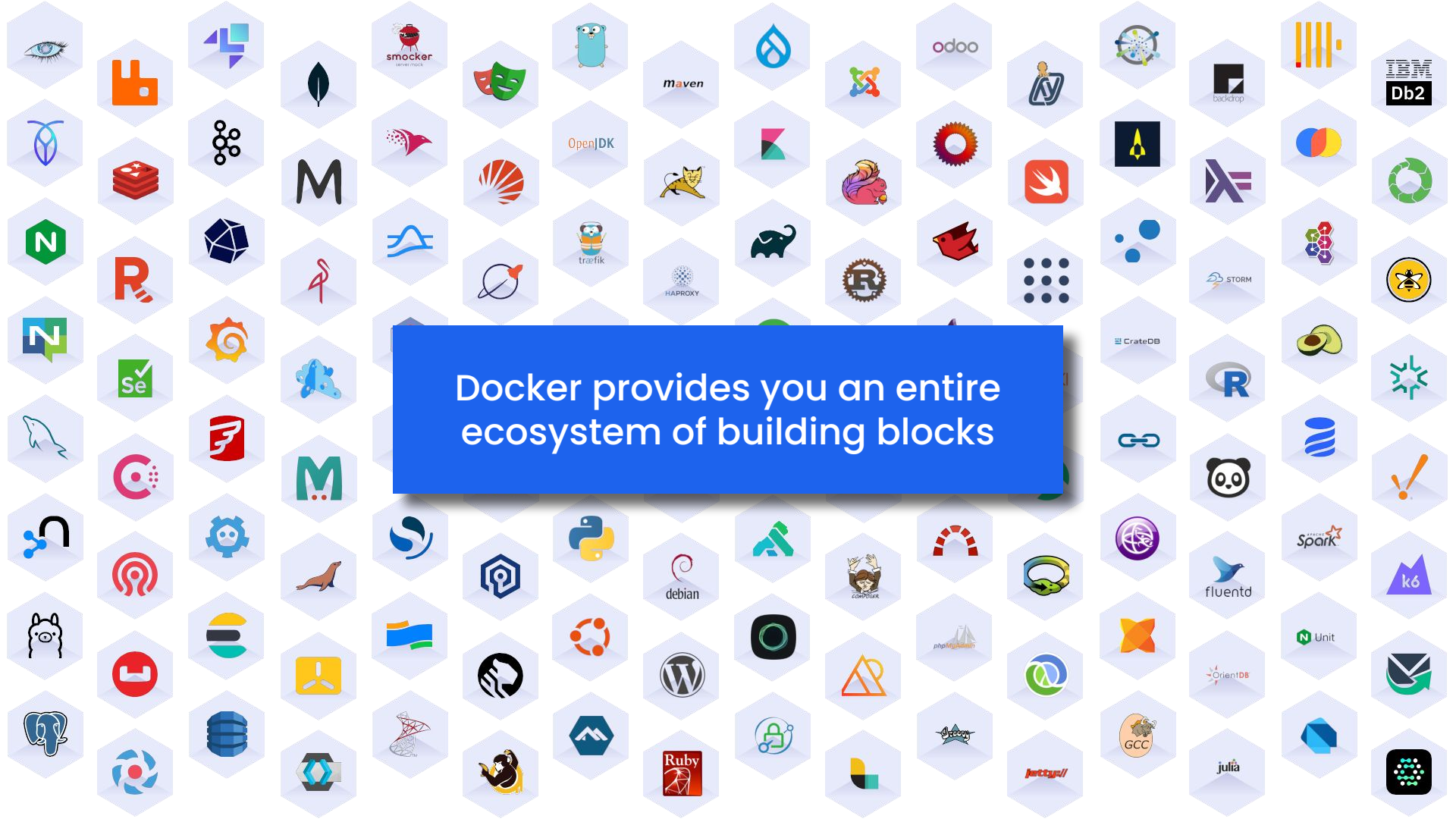
Contains all binaries, files, dependencies,
and configuration needed to run the
containerized process



Registry = Image repository

A centralized location for the hosting and
distribution of container images. Can be
available publicly or privately.



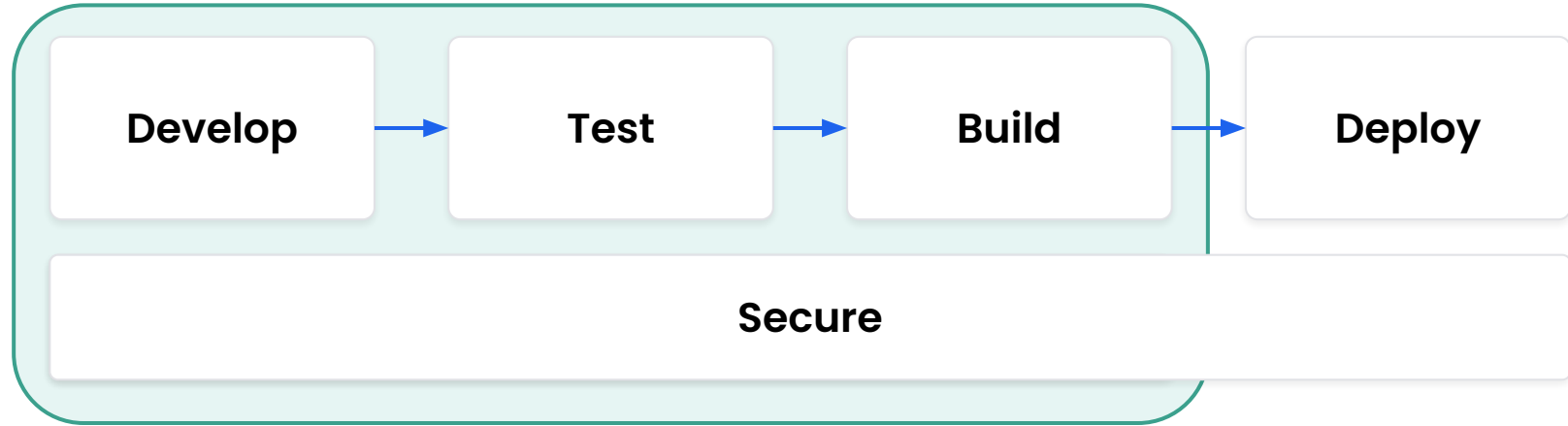


Docker provides you an entire ecosystem of building blocks



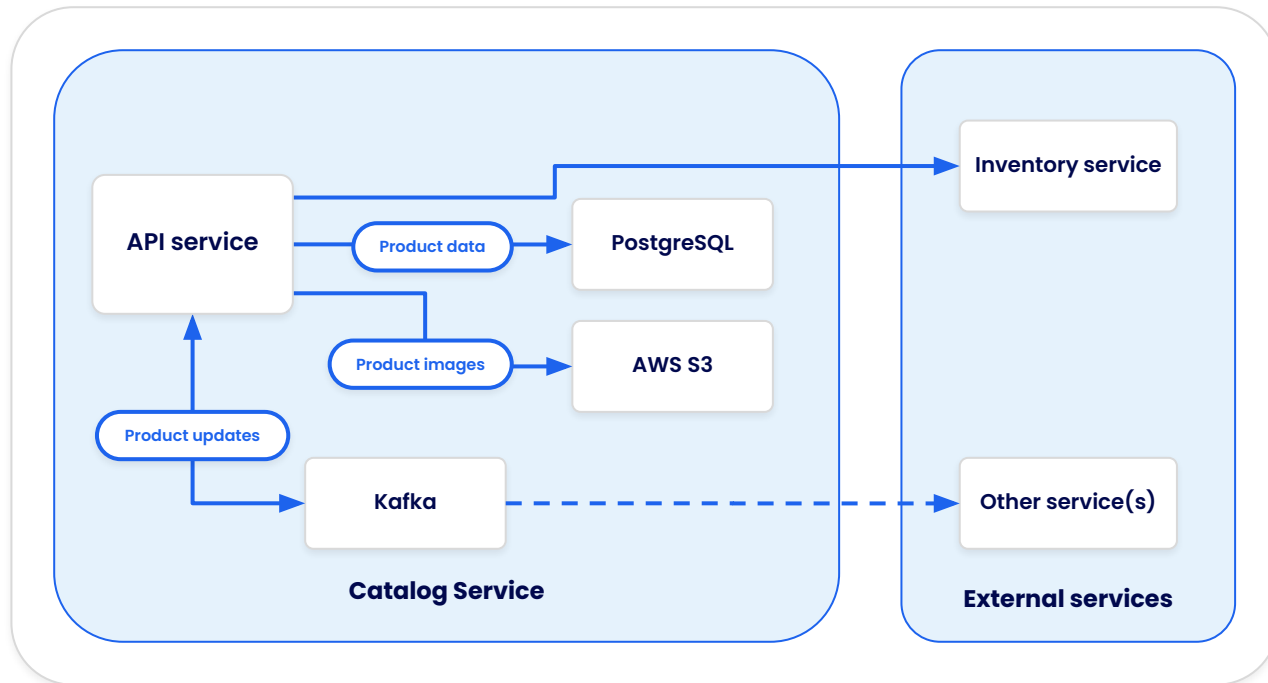
Docker across the SDLC

The SDLC



Sample app = Catalog API

- Data is stored in a **PostgreSQL database**
- Product images stored in a **AWS S3 bucket**
- Inventory data comes from an **external inventory service**
- Product update events are published to a **Kafka cluster**



The ticket

The screenshot shows a Jira ticket interface. The top navigation bar includes links for 'Your work', 'Projects', 'Filters', 'Dashboards', 'Teams', 'Plans', 'Apps', and a 'Create' button. The ticket title is 'Add UPC code to product_created event' with a status of 'In Progress' and a priority of 'High'. The ticket is assigned to 'Moby Dock' and is part of the 'S24: Sprint Awesome!' sprint. The description states: 'As a downstream consumer of catalog events, I would like to have the UPC code included in the product_created event so I don't need to look it up immediately after the event is received.' The activity section shows a comment input field with a 'Pro tip: press M to comment' message. The right sidebar contains sections for 'Details', 'More fields Labels', 'Automation', 'OKRS', and 'Connector for Salesforce'. The ticket was created on December 4, 2024, at 7:55 AM and updated on December 9, 2024, at 9:20 AM.

Devre Software

Sumr

Time

Back

Boar

Caler

List

Form

Issue

Entity

Test s

Add-c

Add s

Projec

Archiv

4000

your work Projects Filters Dashboards Teams Plans Apps Create

Search

Add Objective / DEVR-981

Add UPC code to product_created event

In Progress Actions Improve issue

Details

To be done by None

Assignee Moby Dock

Parent None

Sprint S24: Sprint Awesome!

Development Create branch Create commit

Reporter Moby Dock

More fields Labels

Automation Rule executions

OKRS Profit.co OKRs

Connector for Salesforce Associations

Created December 4, 2024 at 7:55 AM Updated December 9, 2024 at 9:20 AM Configure

Activity

Show: Comments Summarize Newest first

Add a comment...

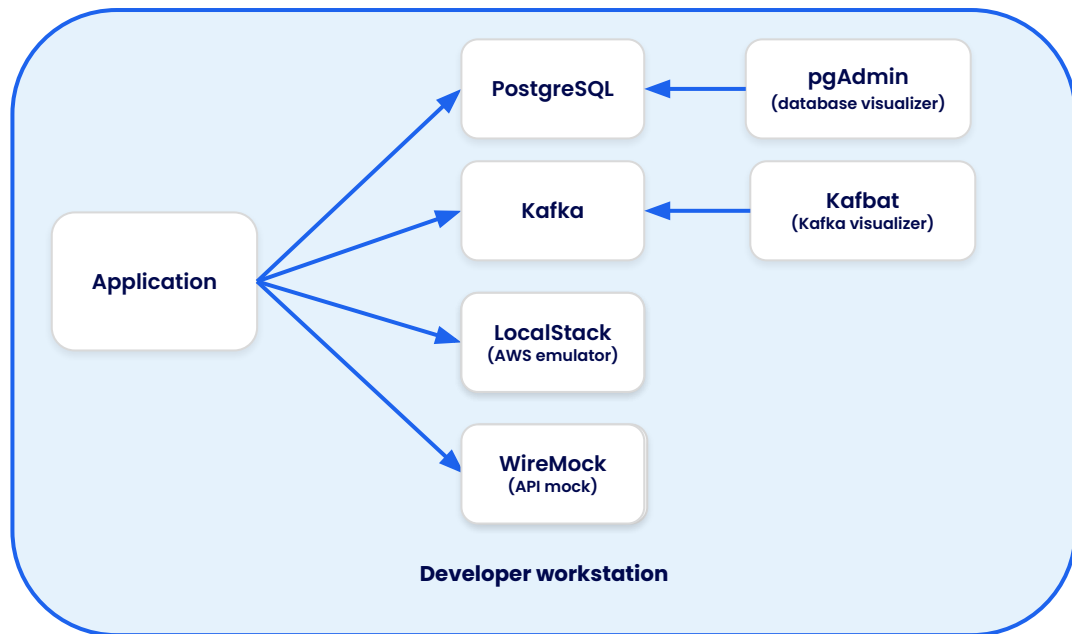
Pro tip: press M to comment





SDLC Stage - Development

The development environment



The development tools and features

Settings
management

Enhanced container
isolation

Registry Access
Management

Image Access
Management

Air-gapped
containers



Docker Engine

Container
networking



Docker CLI/GUI

Port forwarding

Docker Compose

Docker Debug

Volume mounts

Synchronized file
shares



Lab 2

- ✓ Ease to setup and launch the development environment
- ✓ Ability to use additional tools to help troubleshoot and validate changes
- ✓ Ability to use containers in a hybrid setup (app is running natively)

<https://dockerworkshop.com/product-catalog/develop>

Benefits of Docker-based development



Faster onboarding

Less time setting up.
Less time switching projects.



Consistency everywhere

No more "it worked for me".
Less time rolling out env changes.



Project-specific tooling

Less coupling on external services.
Less time troubleshooting issues.



Enterprise ready

Dev workstations kept secure.
Satisfied organizational policies.

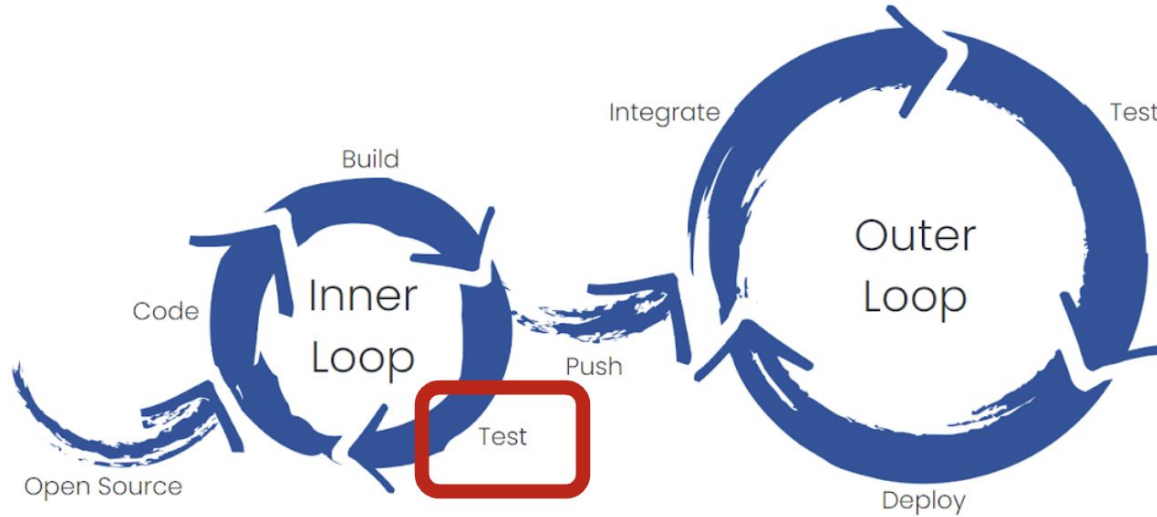




SDLC Stage - Testing

Shifting left with Testcontainers

95% of developers write tests within their inner development loop



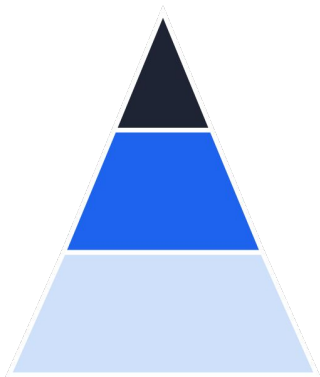
[State of Local Development and Testing](#)

The shapes of testing

Unit

Integration

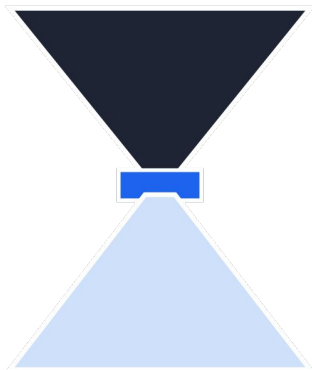
E2E



Pyramid

The long-held ideal

*integration
was too hard*



Hourglass

The trap teams fell into — brittle

*microservices
took over*



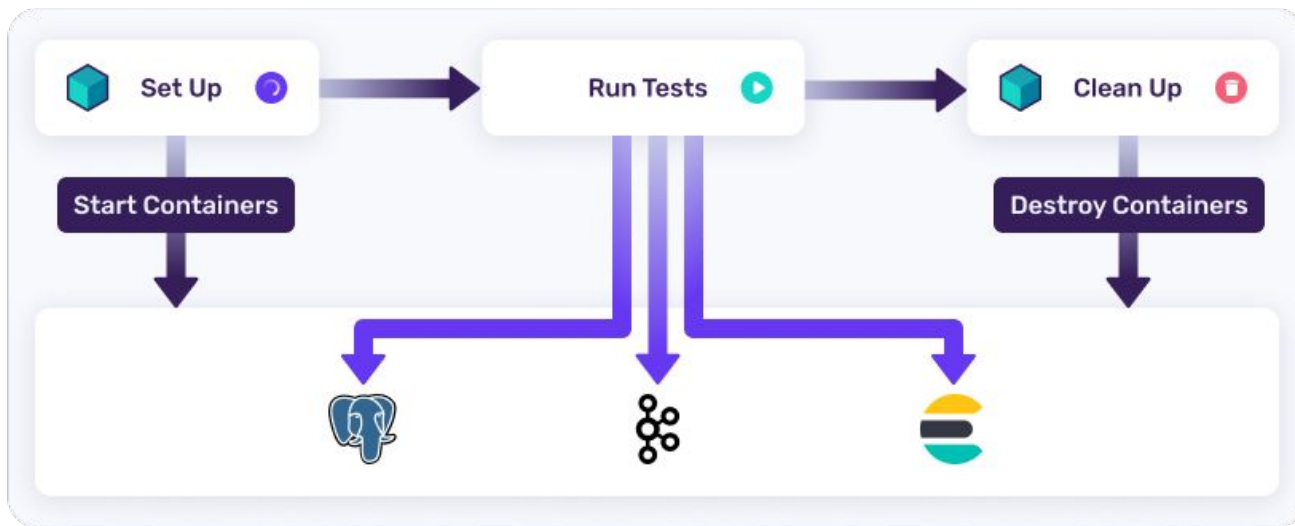
Honeycomb

Modern microservices target

Containers make the integration-heavy honeycomb practical — spin up real dependencies per test, then throw them away.

Introducing Testcontainers

Open source collection of libraries to programmatically control the lifecycle of containers during testing



```
var postgres = new PostgreSQLContainer<>(DockerImageName.parse("postgres:17.2"));
```

Run the containers anywhere!

Run containers locally



**Testcontainers
Desktop** by docker.

Great for local development
and validation

Run with cloud resources



**Testcontainers
Cloud** by docker.

Great for CI pipelines or when more
resources are needed locally



Lab 3

- ✓ Complete control of services during testing
- ✓ Ability to run containers for testing locally or remotely
- ✓ Consistency of test results whether running locally or in CI pipelines

<https://dockerworkshop.com/product-catalog/test/>

Benefits of Docker-based testing



Use real services

Increased deployment confidence.
Reduced code complexity.



Consistent testing envs

Reduced test failures.
Faster test failure resolution.



Dynamic scaling

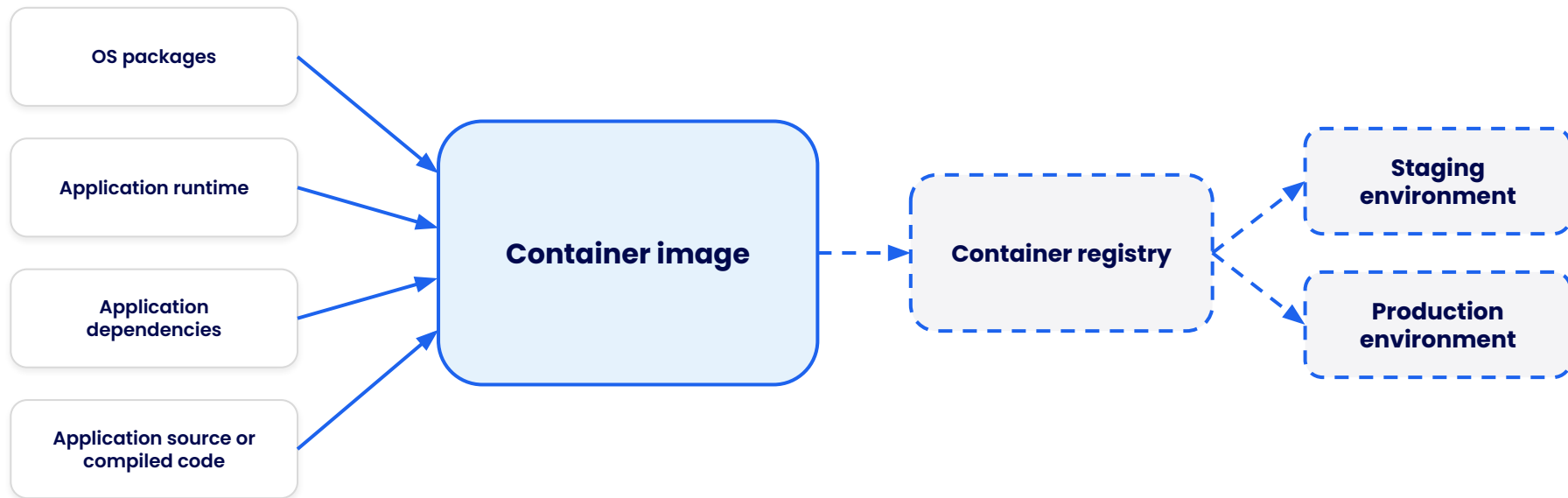
Reduced testing infra costs.
Reduced execution times.





SDLC Stage - Build

Building our application



The build tools and features



Docker Engine

Multi-stage builds

**Registry Access
Management**



Docker CLI/GUI

Build caching

**Image Access
Management**



BuildKit/Buildx

**Multi-architecture
builds**

**Docker GitHub
Actions**

Dockerfiles

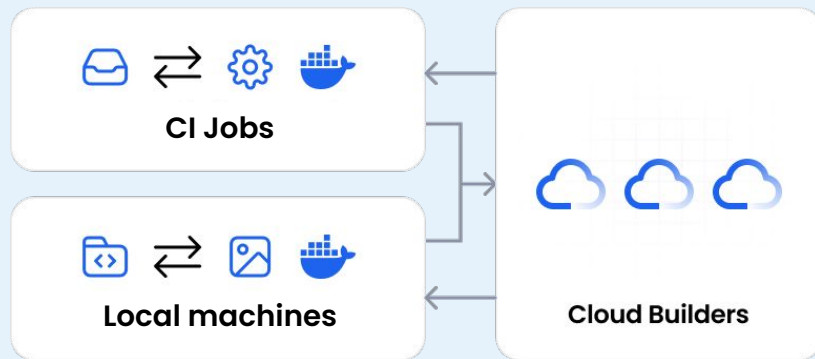
Build secrets

**Build insights
and logs**



Benefits of Build Cloud

- ✓ Shared build cache
- ✓ Native multi-architecture builds
- ✓ Seamless integration with your CI tools
- ✓ Minimal changes in local development



Lab 4

- ✓ Writing a Multi-Stage Dockerfile
- ✓ Demonstrate Docker Layered Caching
- ✓ Building a multi-platform builds

Benefits of Docker-based building



Time savings

Decreased build times.
Decreased machine resources.



Consistency of tooling

Reduced tool sprawl.
Increased build confidence.



Useful insights

Reduced time to recover.
Increased best practice adoption.





SDLC Stage - Secure

The “Software Supply Chain” in Practice

- Software is built from **multiple parts**, dependencies and **open source components**
- All software contains **vulnerabilities** – both known and unknown
- Software development involves **multiple developers**, teams and system
- Components are created by **contributors both inside and outside the organization**
- Attackers target components with vulnerabilities to gain access & insert **malicious code** into an organization’s software



Software Supply Chain Security Challenges

96%

of applications include open source components

70–90%

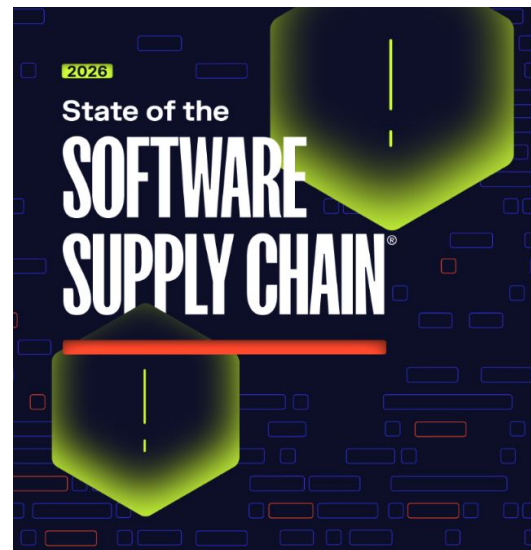
of code in an application is from open source

454,648

new malicious packages identified in 2025 alone

1,23,219

open-source malware packages logged since 2019



Source ~ [The State of Software Supply Chain Report 2026 by Sonatype](#)



Given All That, Every Company Needs to..



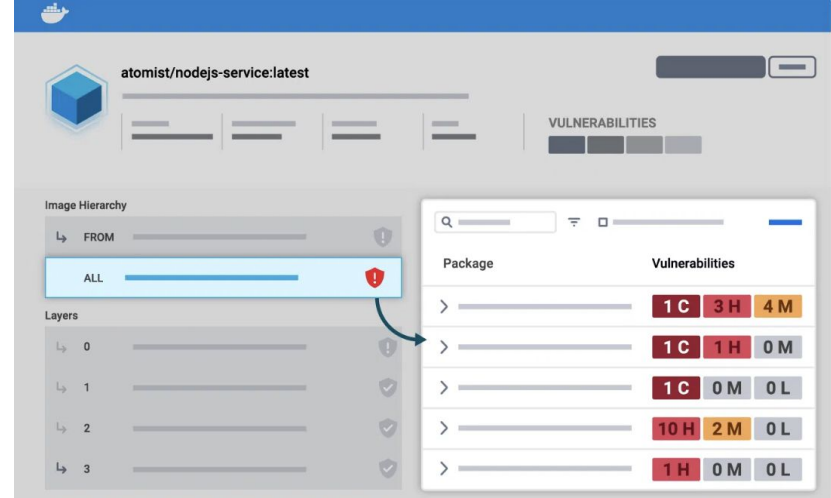
Know what software they're running

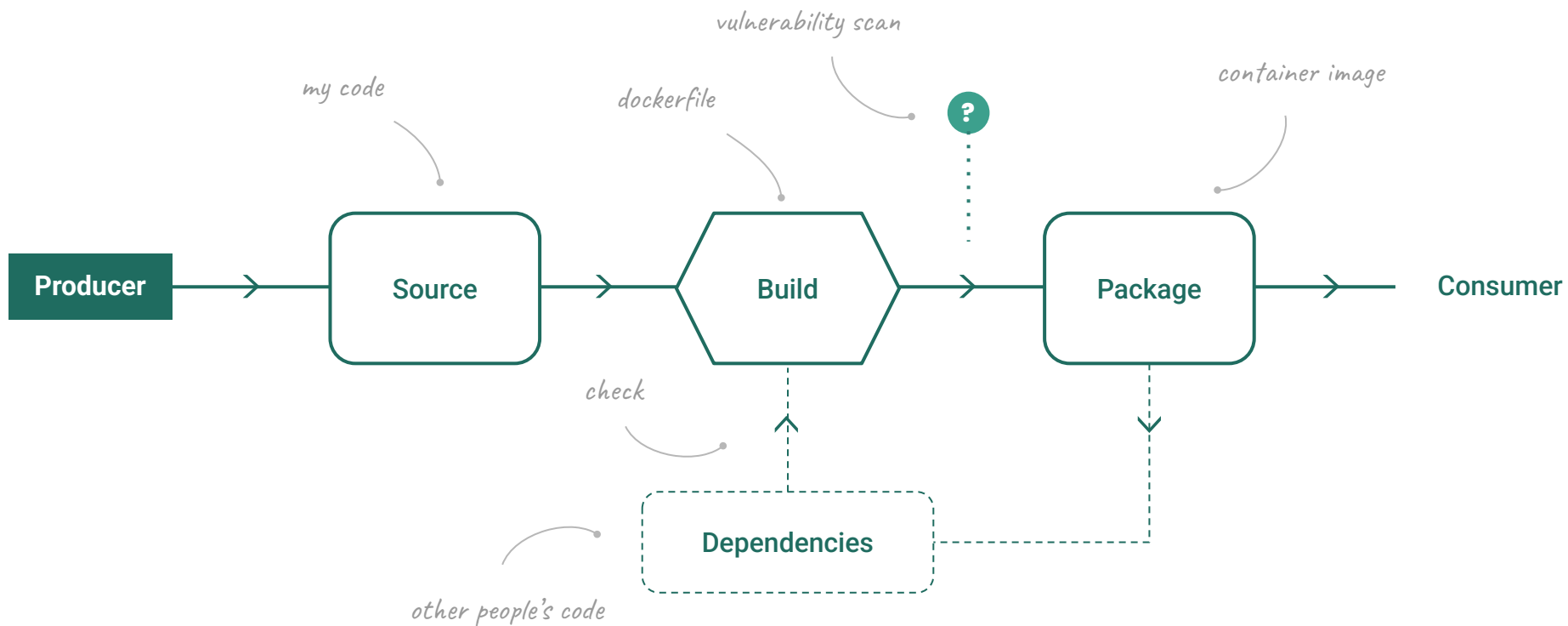


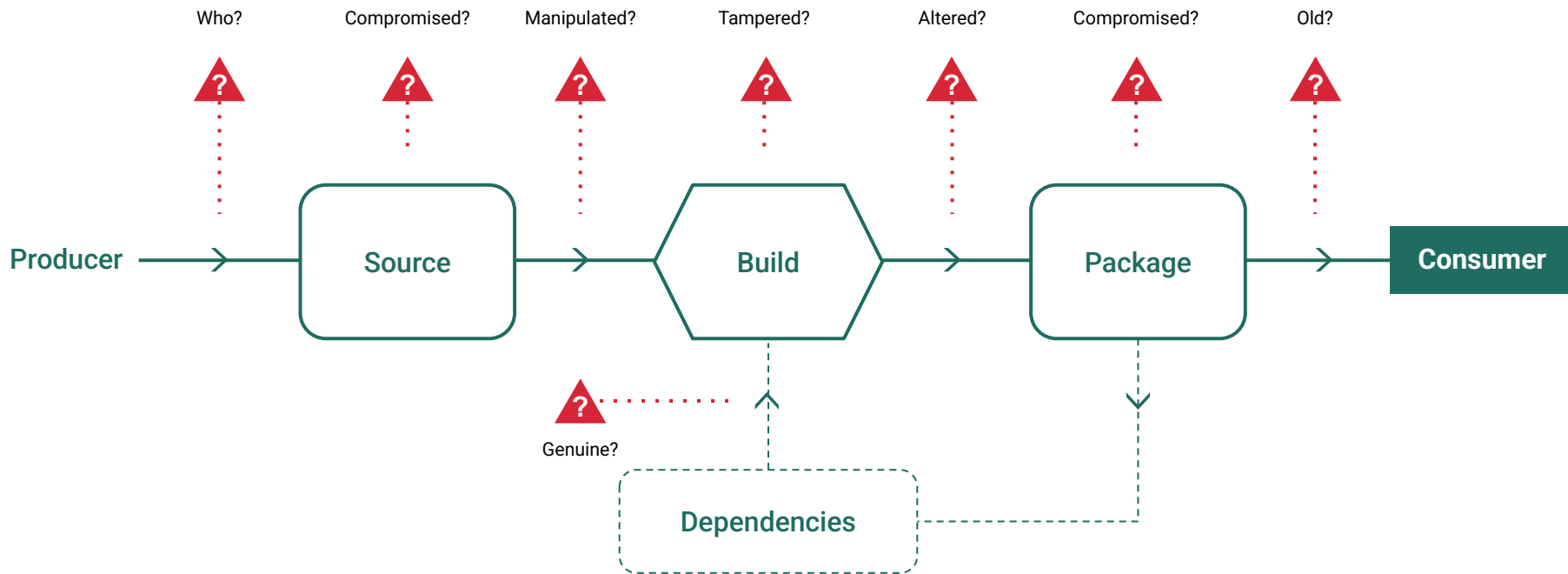
Know what risks that software has



Fix those risks quickly







What's in this software artifact?

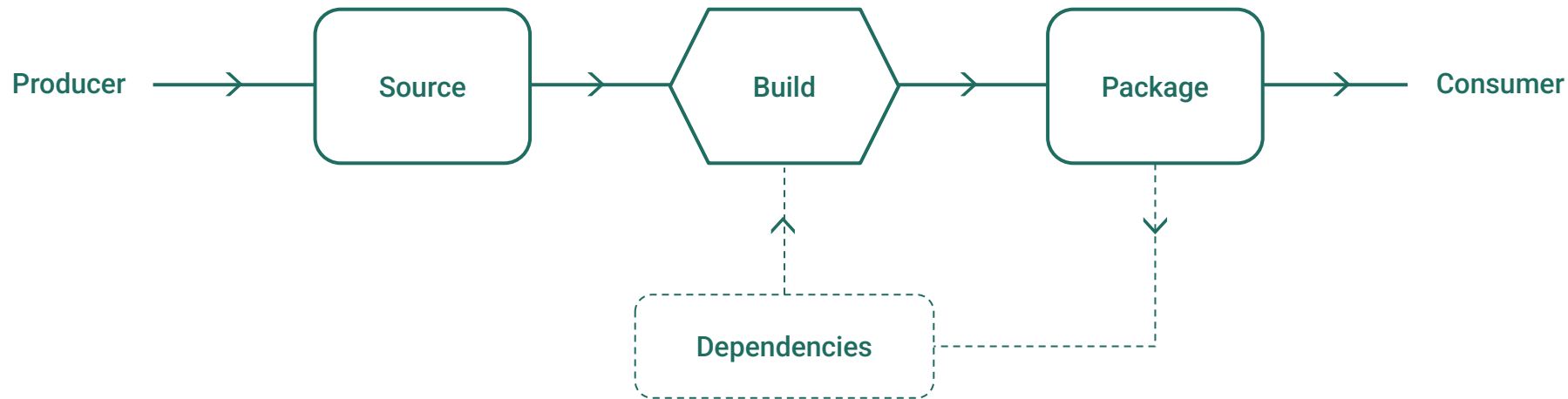
A software bill of materials – **SBOM** – is an *attestation* describing the contents of a software artifact

Where has it come from?

Provenance is an *attestation* about the history of an artifact – where it came from, who produced it, and how

Can I verify the attestation source?

An *attestation* that has been **signed** with a digital signature verifies the source and helps assess trustworthiness



Managing the software supply chain

Developer workflows



Cloud-based



Hybrid | Cloud Dev Environment



Local Dev Laptop

Plan

Setup

Code

Build

Analyze

Debug

Commit

CI

Deploy



Fast builds



Docker Scout

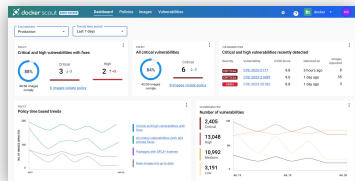


Docker Scout: Actionable insights across the software supply chain



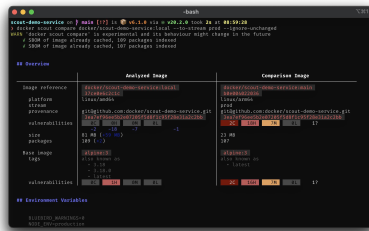
Centralized View

- Unified dashboards that inform next steps to address security issues before they hit production
- Integrations throughout Software Supply Chain: CI/CD, Registry, SCM



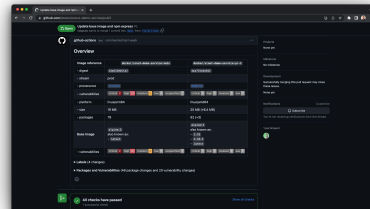
Recommended Workflows

- Policy evaluation to assess outdated base images, quality gates, copyleft licensing, security issues
- Remediation guidance



Secure Development

- In-depth image analysis within Docker Desktop, CLI, and scout.docker.com
- Ongoing feedback loop that informs better decisions at point of development through to production



Empowering Developers with Docker Scout



**Understand your
applications'
dependencies**



**Analyze the
vulnerabilities in
your dependencies**



**Act quickly
with suggested
remediation options**

Lab 5

- ✓ Analyze the Product Catalog container images
- ✓ Fix the vulnerabilities using Scout

Scout CLI Integration with CI Environments



GitHub Actions



GitLab



Jenkins



Azure Pipelines



circleci



ATLASSIAN

Bitbucket

build summary

Vulnerabilities of ajeetraina/scout-demo-service:main

Image Reference ajeetraina/scout-demo-service:main

digest	sha256:f93c2de116df7ad0eacc154537c968812b14e8a17544f7d3004449515583bd35
vulnerabilities	critical 2 high 18 medium 7 low 0 unspecified 1
platform	linux/arm64
size	19 MB
packages	79

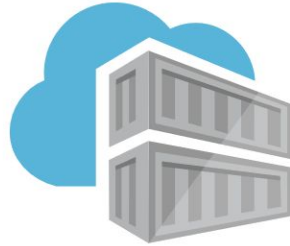
Base Image alpine:3.14.1

▶ C 1 H 4 M 5 L 0	openssl 1.1.1k-r0 (apk)
▶ C 1 H 1 M 0 L 0	zlib 1.2.11-r3 (apk)
▶ C 0 H 10 M 2 L 0 U 1	busybox 1.33.1-r3 (apk)
▶ C 0 H 1 M 0 L 0	qs 6.7.0 (npm)
▶ C 0 H 1 M 0 L 0	libretls 3.3.3p1-r2 (apk)
▶ C 0 H 1 M 0 L 0	express 4.17.1 (npm)

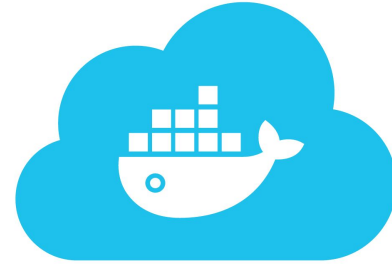
Job summary generated at run-time

<https://scout.docker.com>

Scout CLI Integration with Container Registries



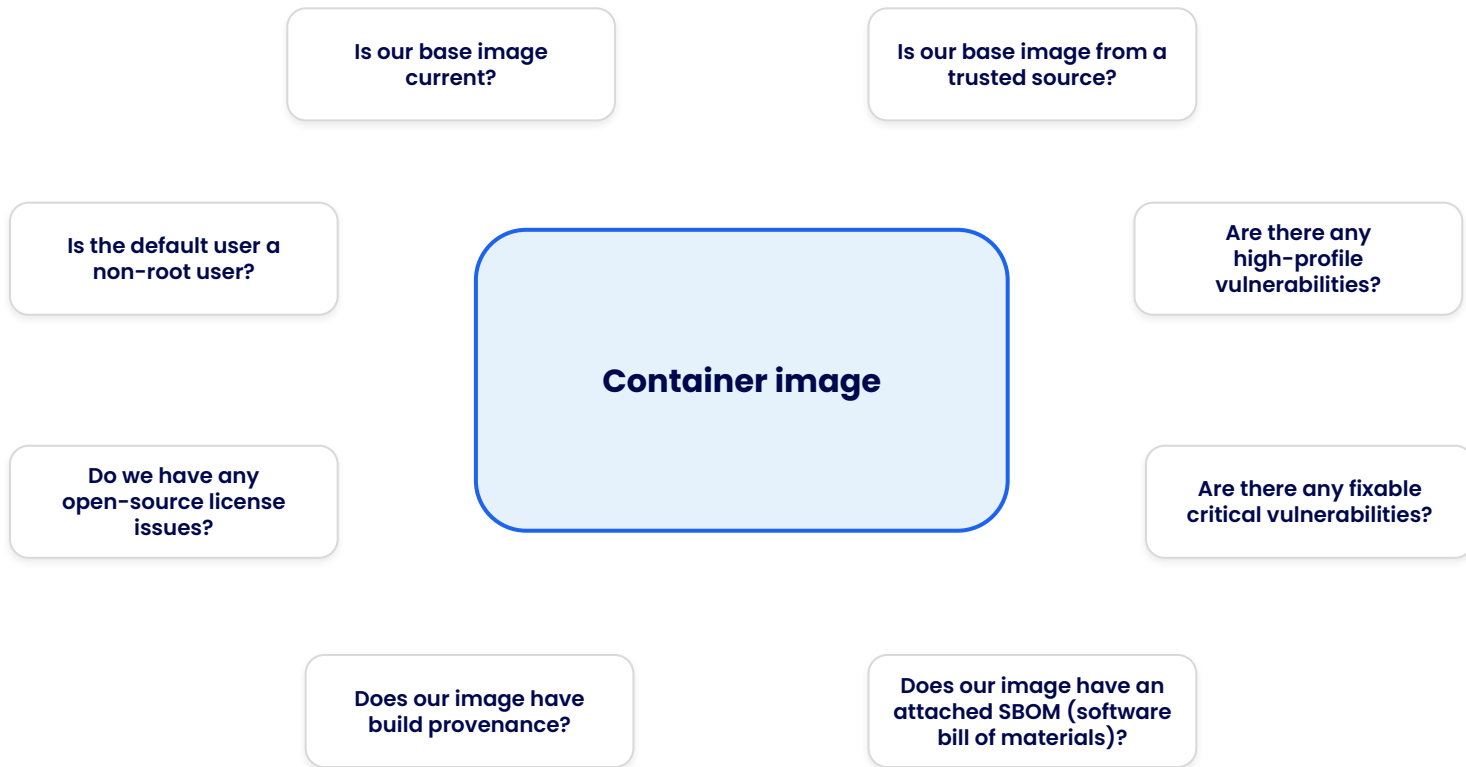
Azure Container Registry



Amazon ECR

<https://scout.docker.com>

Example policies



The secure tools and features



CLI tooling

Build provenance
generation

Environment
recording



GUI integrations

Change notifications

Image comparison

Policy analysis

Remediation
guidance

Official GitHub
Actions

SBOM generation

Vulnerability
assessment

External registry
integrations





Challenges with Reactive Security

Challenges with Reactive Security

Current Approach:

Find › Fix › Repeat

✗ Reactive Approach

"Scan and Fix"



Pull unverified base image (e.g., alpine:latest)



Scan reveals **247 vulnerabilities**
(43 Critical, 89 High)



Developer spends **3 days** researching fixes



Update packages, rebuild, rescan
Still **189 vulnerabilities** remain



Cycle repeats... Security blocks deployment

Result:

- ⌚ Slow deployment cycles
- 😡 Developer frustration
- ⚖️ Security vs. velocity trade-off
- ♻️ Always playing catch-up

Pain Points & Solutions



I need a way to limit the attack surface in my images



I can't keep up with all of the vulnerabilities and remediations impacting my applications



I need a single source of approved container dependencies



I need production ready images that I can just deploy



I need a faster/more sustainable way to achieve regulatory compliance



Minimal images w/ no shell or package management



Automated pipeline delivery of patched vulnerabilities



Sizeable catalog of base images and application images



Support for customizations and deployment artifacts (e.g. Helm charts)



FIPS and STIG supported images

We want your development team to **focus
on building features, not chasing CVEs**



DHI Approach

"Start Secure"

✗ Reactive Approach

"Scan and Fix"



Pull unverified base image (e.g., alpine:latest)



Scan reveals **247 vulnerabilities**
(43 Critical, 89 High)



Developer spends **3 days** researching fixes



Update packages, rebuild, rescan
Still **189 vulnerabilities** remain



Cycle repeats... Security blocks deployment

Result:

- 🕒 Slow deployment cycles
- 😡 Developer frustration
- ⚖️ Security vs. velocity trade-off
- ♻️ Always playing catch-up

✓ Proactive Approach

"Start Secure"



Select **Docker Hardened Image**
(pre-hardened, verified)



Scan reveals **<10 vulnerabilities**
(0 Critical, 2 High)



Clear remediation path provided
Focus on your application code



Build with confidence
Compliance built-in (SBOM, signatures)



Deploy rapidly
Auto-updated by Docker

Result:

- ⚡ Fast, secure deployments
- 😊 Happy developers
- 🚀 Security enables velocity
- 🛡️ Stay ahead of threats

What are Docker Hardened Images (DHI)?

The left screenshot displays the 'Hardened images' landing page. It features a green banner with the Docker logo and the text 'Hardened NodeJS – Secure, Compliant, Ready' and 'Built for security. Minimal CVEs, verified provenance, always up-to-date.' Below this is a grid of application stacks including Dockerflow, TensorFlow, TensorFlow, TensorFlow, and TensorFlow.

The right screenshot displays the 'Manage hardened images' dashboard. It includes a 'Security posture overview' chart showing the number of images by vulnerability status (Low, Medium, High, Critical). It also features a 'Built hardened images' table with columns for Applications, Type, Base image, Release version, Status, and Released.

Applications	Type	Base image	Release version	Status	Released
Node.js API Service	Container Image	Ubuntu Minimal 20.04	v16.16.0	In-progress	Pending
Redis Cache Cluster	Helm Chart	Alpine 3.16	v7.0.9	Released	Jan 23, 2023
PostgreSQL DB	Container Image	Debian 11	v15.2	Released	Dec 15, 2024
Apache Kafka	Helm Chart	Red Hat UBI 8	v3.0.1	Released	Jan 23, 2023
Nginx Reverse Proxy	Container Image	Ubuntu 22.04	v1.25.0	Released	Jan 23, 2023
TensorFlow Inference	Container Image	Debian 11	v2.11.0	Released	Jan 23, 2023
Minio S3	Helm Chart	Amazon Linux 2	v6.0.4	Released	Jan 14, 2023
Jaeger CI/CD Server	Container Image	Ubuntu Minimal 20.04	v2.37.1	Deprecated	Jan 21, 2023
Redis Cache Cluster	Helm Chart	Alpine 3.16	v7.0.9	Released	Jan 23, 2023
PostgreSQL DB	Container Image	Debian 11	v15.2	Released	Dec 15, 2024
Apache Kafka	Helm Chart	Red Hat UBI 8	v3.0.1	Released	Jan 23, 2023
Nginx Reverse Proxy	Container Image	Ubuntu 22.04	v1.25.0	Released	Jan 23, 2023

- ✓ Minimal, secure and production-ready container base and application images maintained by Docker Team
- ✓ Designed to reduce vulnerabilities and simplify compliance
- ✓ DHI Integrates easily into your existing Docker-based workflows with little to no retooling required
- ✓ Customizations supported out of the box include certificates, startup scripts, customisation and additional packages

<https://www.docker.com/products/hardened-images/>



Ultra-Minimal Footprint with
Near-Zero CVEs

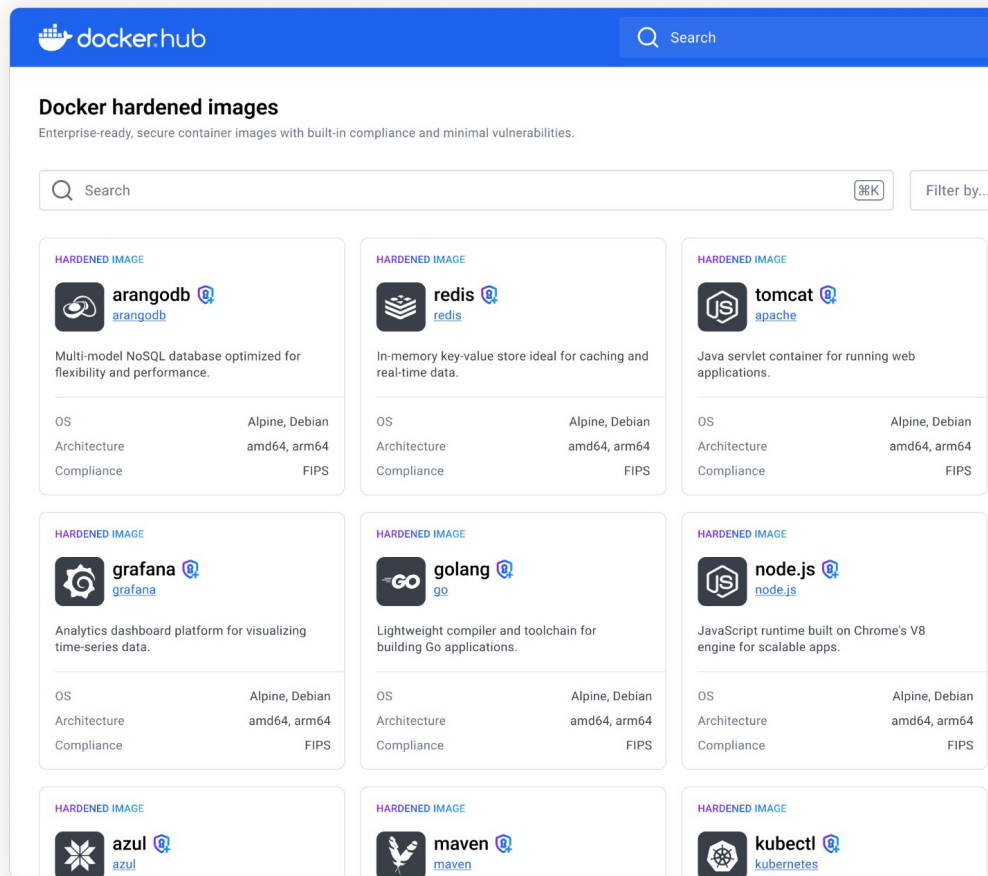


7-Day Remediation for Critical &
High CVEs, SLA-Guaranteed



Built in provenance, SLSA
compliance, SBOMs

<https://hub.docker.com/hardened-images/catalog>



Lab 6

- ✓ Migrate the Node app to DHI
- ✓ Compare the images



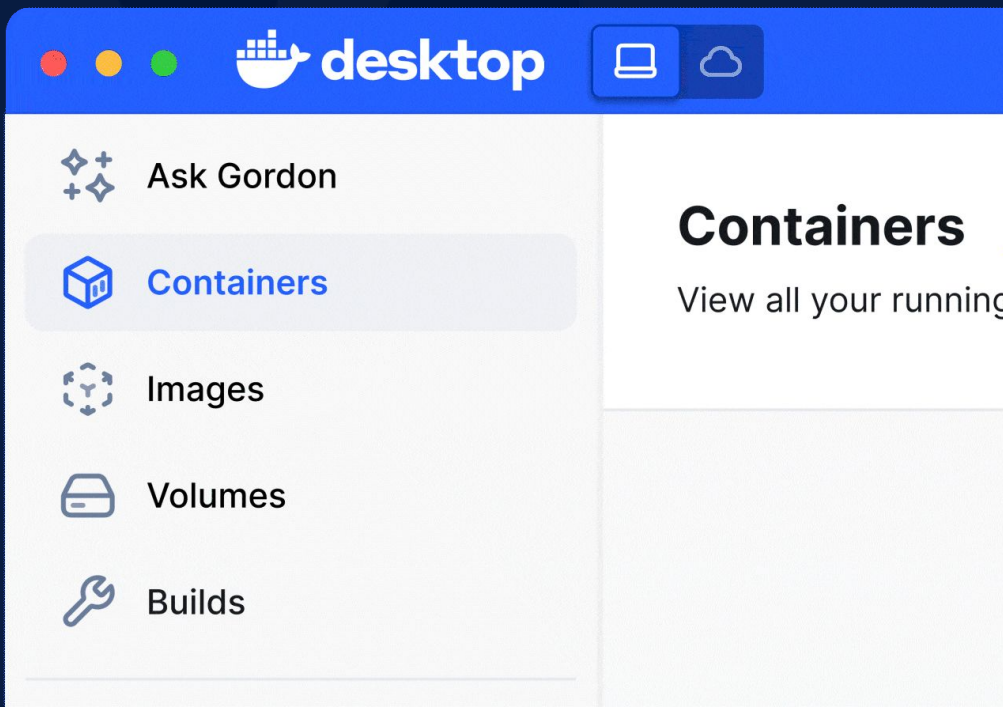
SDLC Stage - Deploy



offload

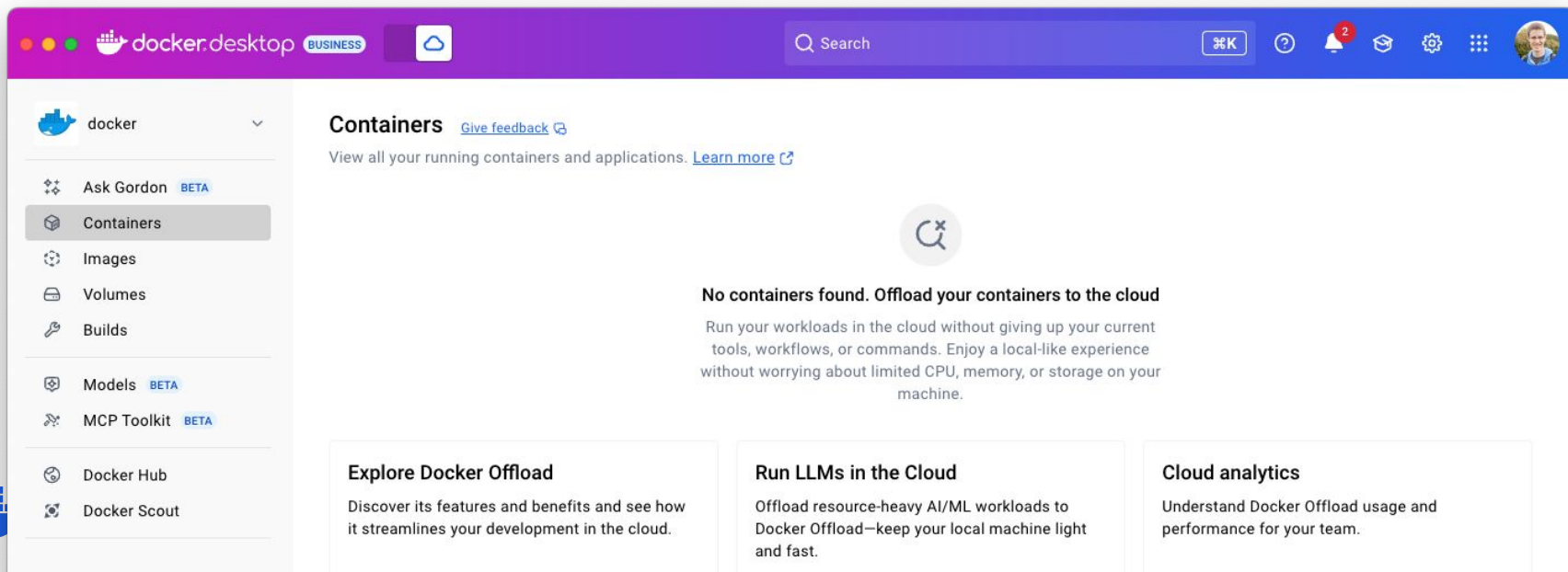
BETA

- ✓ Cloud-based environment that mirrors your local Docker Desktop experience
- ✓ Build and run container images faster by offloading resource-intensive tasks to the cloud
- ✓ GPU acceleration for AI/ML development and Testing
- ✓ Resource-intensive builds that exceeds local machine capabilities



Docker Offload

Offload to the Cloud with GPU Acceleration



Lab 7



Enable Docker Offload



Run GPU accelerated workloads



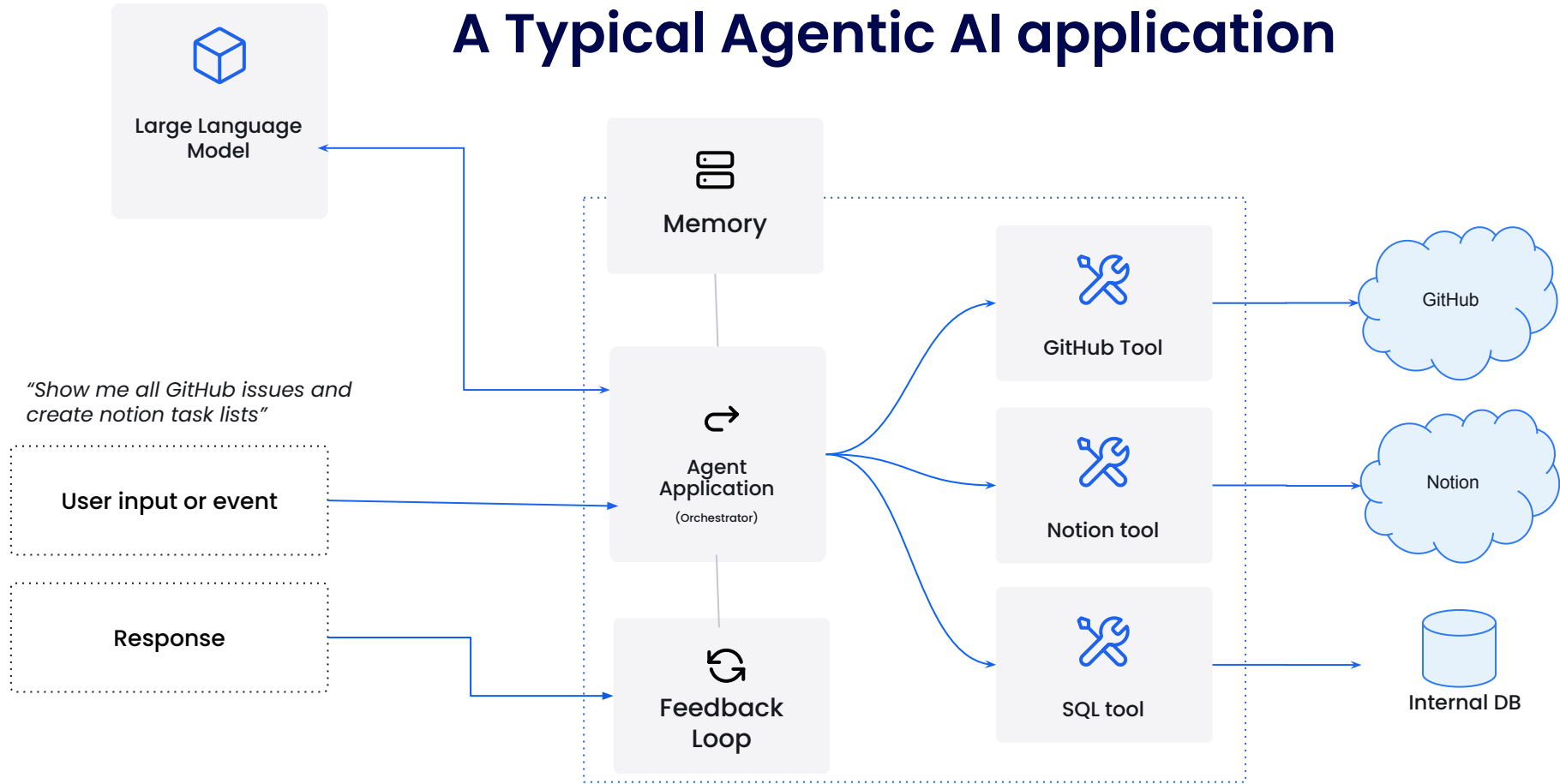
Stop Docker Offload

<https://dockerworkshop.com/lab7/overview>



Docker and AI

A Typical Agentic AI application



Agentic applications need three things



Models



Tools



Code





Models

The Docker Model Runner

Run models next to your other containerized services using the tools you're already using

```
> docker model --help
Usage: docker model COMMAND
```

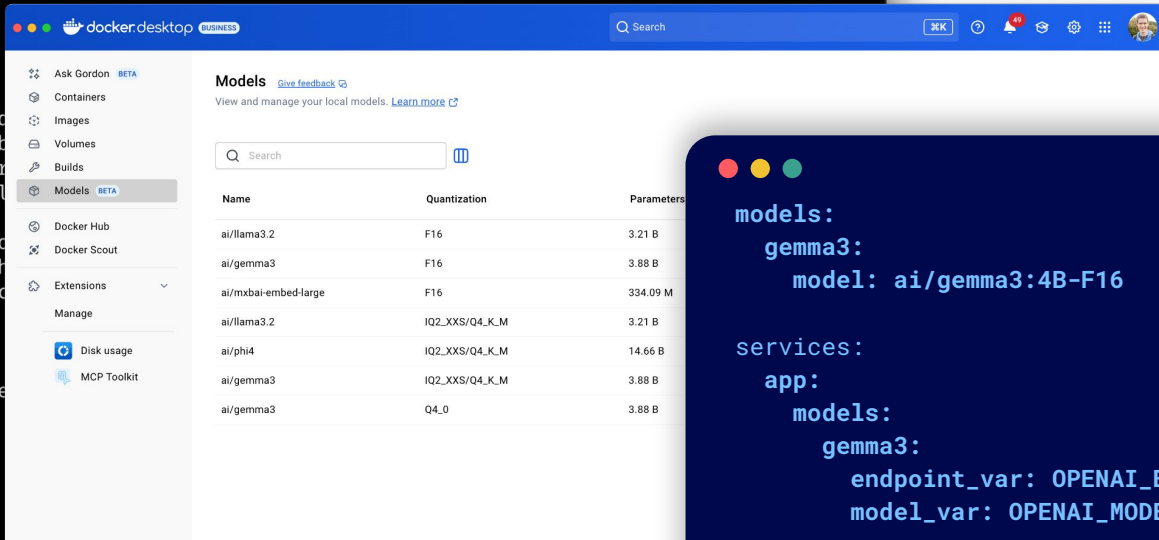
Docker Model Runner

Commands:

inspect	Display detailed
list	List the availab
logs	Fetch the Docker
pull	Download a model
push	Upload a model
rm	Remove models do
run	Run a model with
status	Check if the Doc
tag	Tag a model
version	Show the Docker

```
Run 'docker model COMMAND --he
```

```
~
> |
```



compose.yaml

```
models:
  gemma3:
    model: ai/gemma3:4B-F16

services:
  app:
    models:
      gemma3:
        endpoint_var: OPENAI_BASE_URL
        model_var: OPENAI_MODEL
```


Introducing Docker Model Runner

A GPU-accelerated LLM runner that enables developers to run AI models locally



Enhance development velocity

- Docker-built LLM runner optimized for Apple Silicon & GPU acceleration
- Seamless UX, making it easier to run AI models locally
- Inference Engine in Docker Desktop



Run AI Models Locally with Ease: No complex setup. Just pull a model and run it.



Scalable: Designed to integrate with containerized workflows, supporting both local development and cloud inference.



Optimized for Apple Silicon as well as Windows 11 with NVIDIA GPUs

Traditional GenAI Development

Model Library +
Runtime Config + GPU
Setup

Manual Integration

Fragmented Workflow

Cloud APIs + Custom Scripts +
Storage

- ✗ Manual tool integration
- ✗ Hardware configuration headaches
- ✗ Multiple disconnected workflows
- ✗ High cloud API costs
- ✗ Data privacy risks

Docker Model Runner

Docker Desktop

Model Runner

Unified Docker
Workflow

- ✓ Simple Docker CLI
- ✓ Native GPU acceleration
- ✓ Unified container workflow
- ✓ Cost-effective local runs
- ✓ Data stays on your machine



\$ docker model

By using this new CLI, developers can:

- Pull models from registries (e.g Docker Hub)
- Run models locally with GPU acceleration
- Integrate models into their development workflows
- Test GenAI applications during development without relying on external APIs

```
ajeetsraina hello-genai → (main) 23:13 docker model
Usage: docker model COMMAND

Docker Model Runner

Commands:
  inspect  Display detailed information on one model
  list     List the available models that can be run with the Docker Model Runner
  pull     Download a model
  rm       Remove a model downloaded from Docker Hub
  run      Run a model with the Docker Model Runner
  status   Check if the Docker Model Runner is running
  version  Show the Docker Model Runner version

Run 'docker model COMMAND --help' for more information on a command.
```



Pulling the Model from registries

```
docker model pull ai/llama3.3
```

```
...
```

```
Model ai/llama3.3 pulled successfully
```

AI Model Management Interface

✦ Ask Gordon BETA

📦 Containers

🔄 Images

💾 Volumes

🔧 Builds

📦 Models BETA

🌐 Docker Hub

🔍 Docker Scout

🔗 Extensions

Models

[Give feedback](#)

View and manage your local models. [Learn more](#)

🔍 Search

📋

Name	Quantization	Parameters	Size	Actions
ai/llama3.2	Q8_0	1.24 B	1.22 GiB	▶ 🗑️
ai/gemma3	F16	3.88 B	7.23 GiB	▶ 🗑️
ai/mxbai-embed-large	F16	334.09 M	638.85 MiB	▶ 🗑️
ai/phi4	IQ2_XXS/Q4_K_M	14.66 B	8.43 GiB	▶ 🗑️

Model Runner Integration with Docker Compose

Introducing a new service type in Compose called **provider**



Run AI model as part of multi-container app

- Introduced for the first time in Compose 2.35 or later
- Requires Docker Desktop 4.41+
- Uses the **model** top-level element in your compose file



Pull and Run AI models
Locally with Ease



Define and Run
AI-powered
applications alongside
your other services



Support for
Apple Silicon
as well as
Docker desktop
for Windows
with NVIDIA
GPUs

```
services:
  chat-app:
    image: my-chat-app
    models:
      - llm

models:
  llm:
    model: ai/smollm2
```



Lab 8

- ✓ Enable Model Runner
- ✓ Integrate Chatbot Service
- ✓ Access the ChatBot App

<https://www.dockerworkshop.com/lab4/projects/catalog-chatbot/>

Agentic applications need three things



Models



Tools

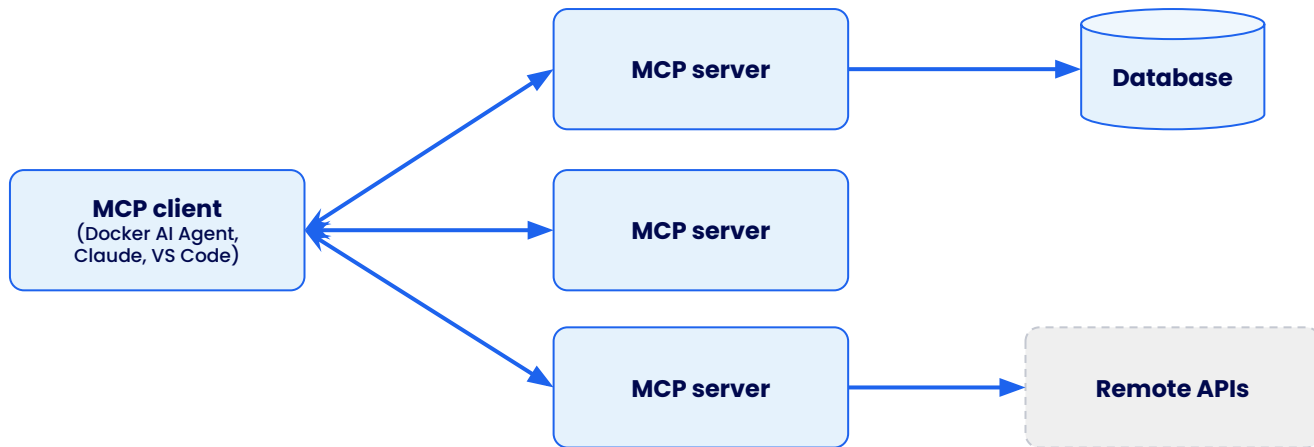


Code



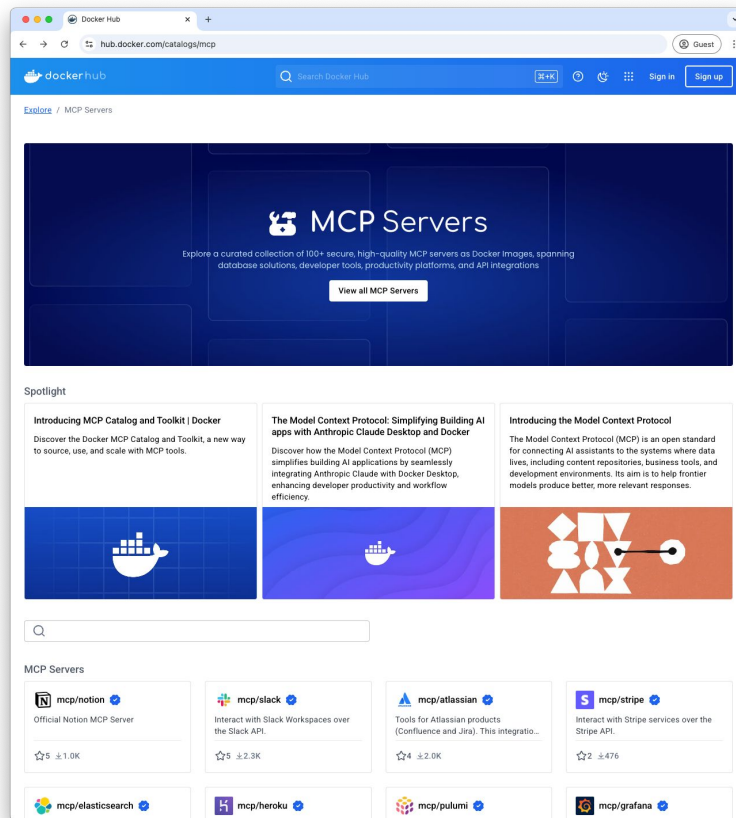
Model Context Protocol (MCP)

- "Like a USB-C port for AI applications"
- Provides the ability to provide context to LLMs
 - a. Tools, prompts, resources, and more



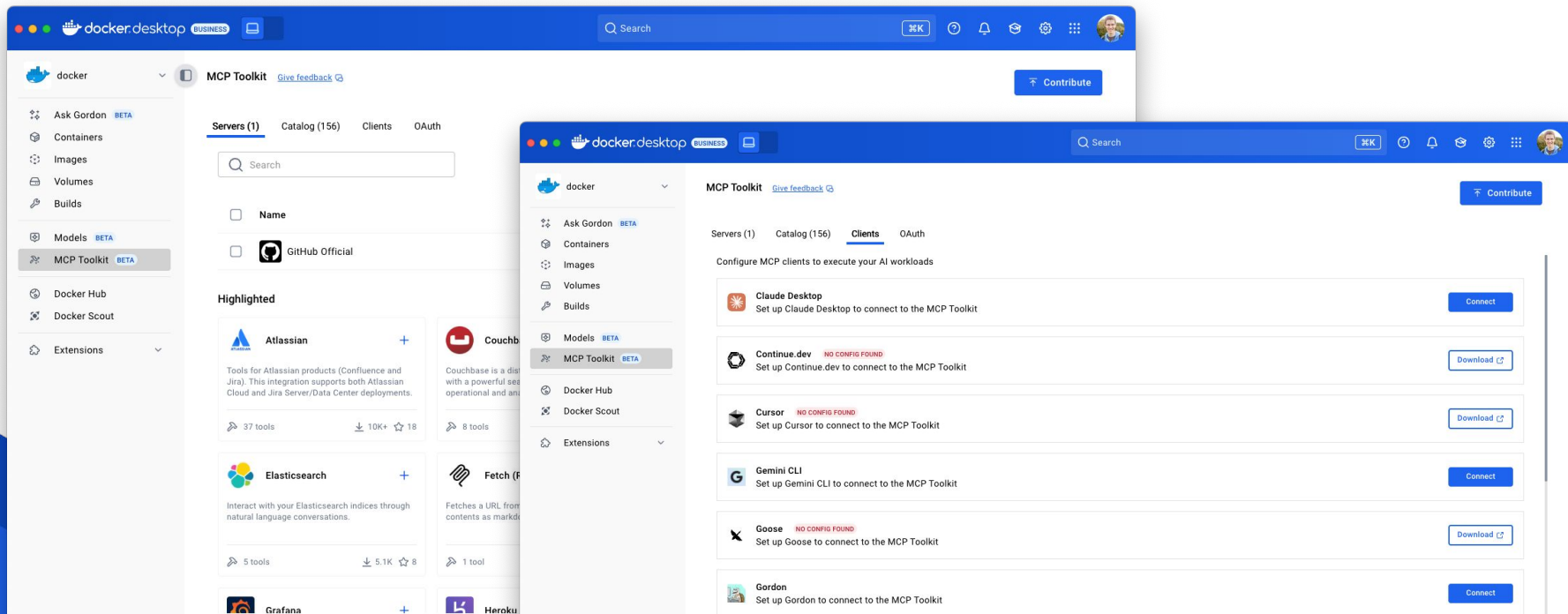
The Docker MCP Catalog

- Collection of 300+ containerized MCP servers
 - No longer worry about runtime setup and host isolation
- Includes Stripe, GitHub, Elastic, Heroku, Grafana Labs, and more
- hub.docker.com/catalogs/mcp



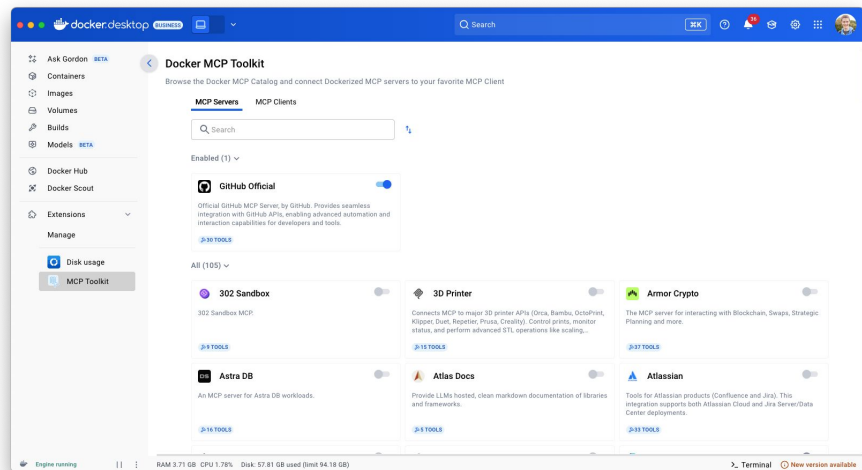
The MCP Toolkit

Convenient packaging to securely connect MCP servers to your Desktop agentic apps



The Docker MCP Toolkit

- Launch MCP servers with minimal effort
- Credential storage and secure injection into containers
- Single connection endpoint for all of your MCP clients





How to Manage Lifecycle of containerised MCP Servers?

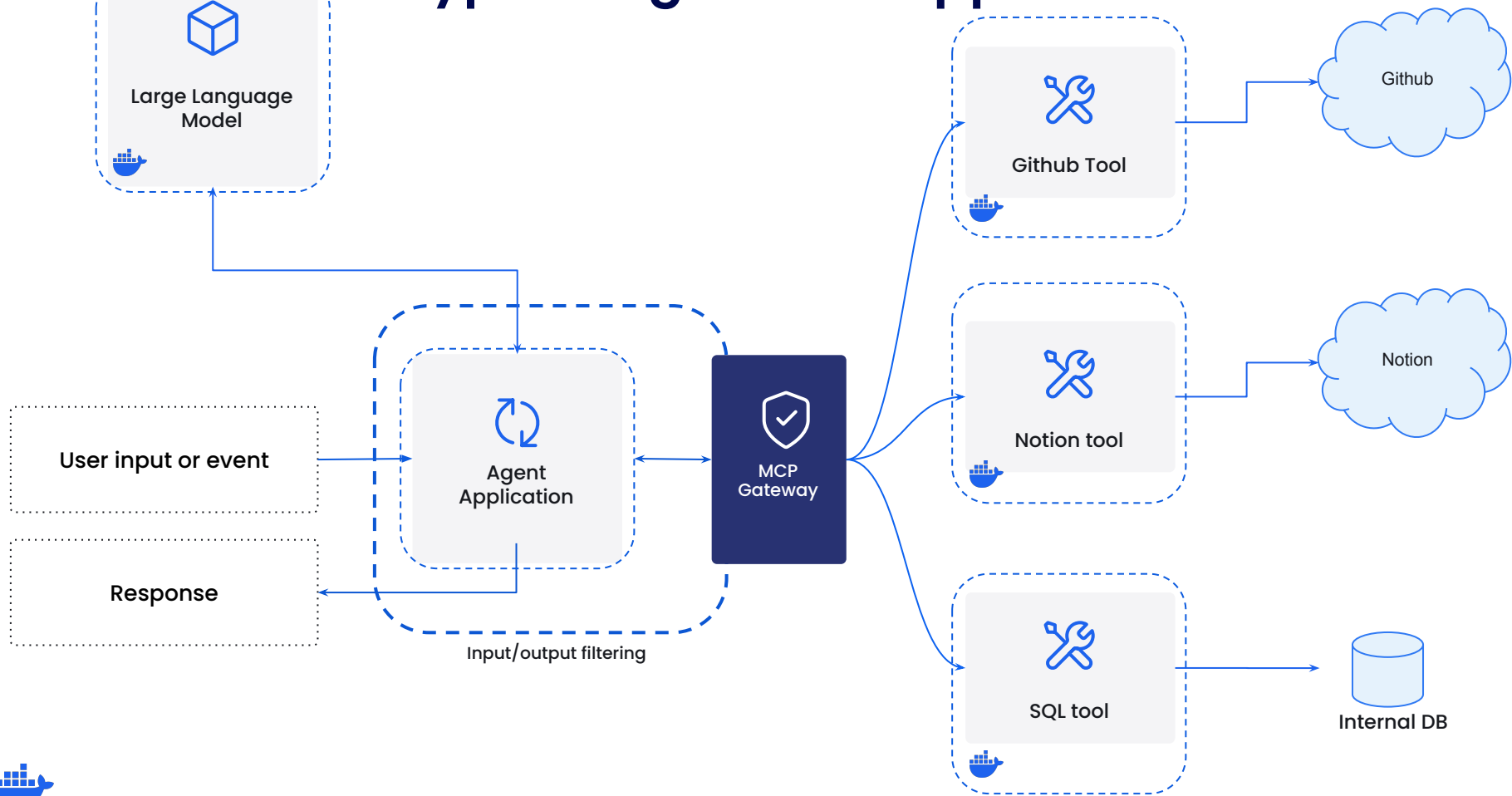
The MCP Gateway

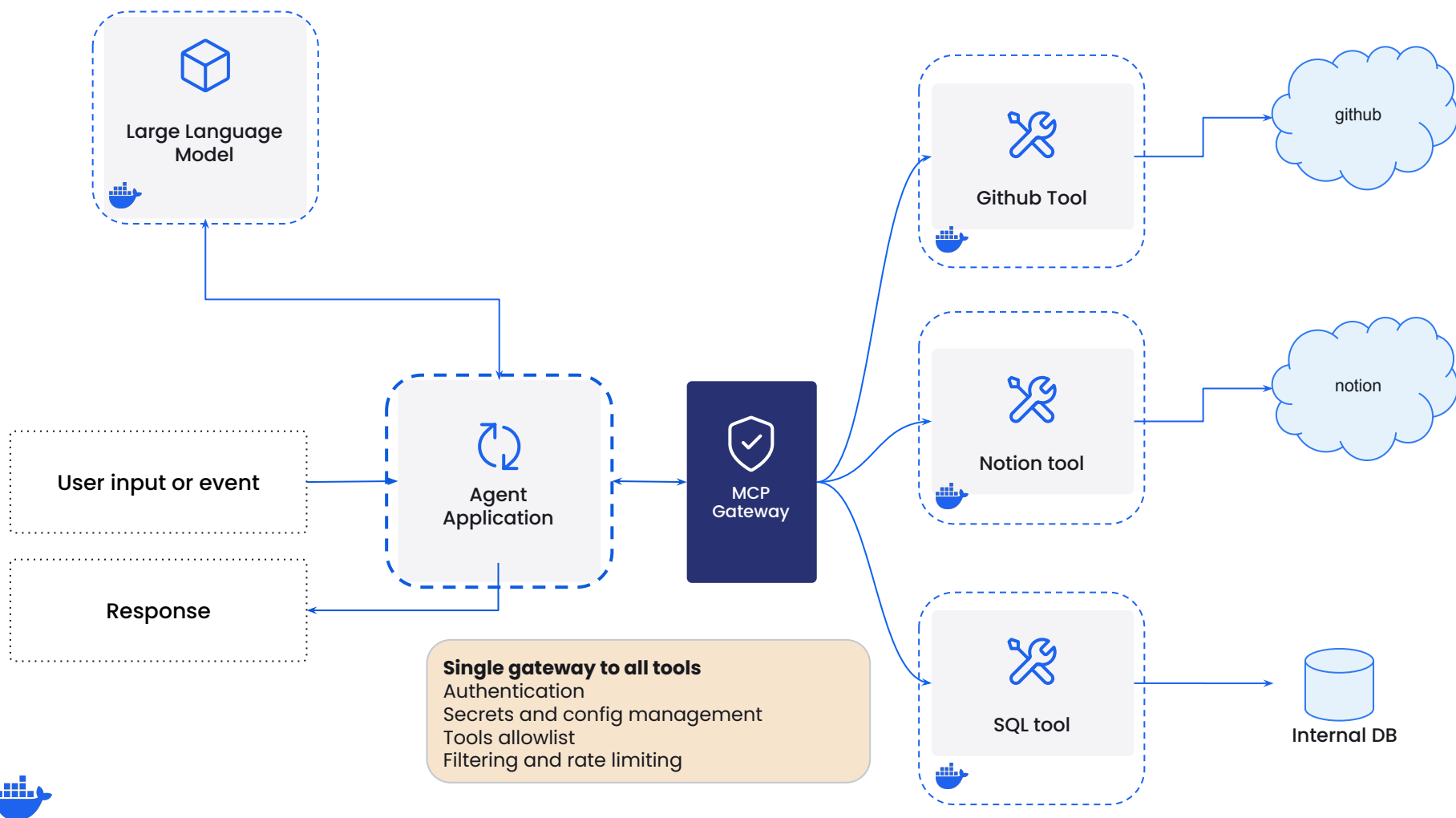
Run containerized MCP servers safely and securely directly in your application stack

```
compose.yaml

services:
  mcp-gateway:
    image: docker/mcp-gateway:latest
    use_api_socket: true
    command:
      - --transport=sse
      - --servers=duckduckgo
      - --tools=search,fetch_content
  app:
    ...
    environment:
      MCP_ENDPOINT: http://mcp-gateway:8811/sse
```

A Typical Agentic AI application





Docker MCP Gateway



Orchestrate and manages MCP servers securely across Dev and Prod



Secure handling of API keys and credentials via Docker Desktop



One gateway endpoint centralizes configuration, credentials, and access control for all MCP servers



Built-in logging and call tracing capabilities



Run in isolated Docker containers with restricted privileges, network access, and resource usage



```
docker run -d \  
    -p 8811:8811 \  
    --restart=always \  
    --name=mcp-gateway \  
    --use-api-socket \  
    -v $HOME/.docker/mcp:/mcp:ro \  
    docker/mcp-gateway \  
    --catalog=/mcp/catalogs/docker-mcp.yaml \  
    --config=/mcp/config.yaml \  
    --registry=/mcp/registry.yaml \  
    --tools-config=/mcp/tools.yaml \  
    --secrets=docker-desktop \  
    --watch=true \  
    --transport=sse \  
    --port=8811
```


Calling MCP Gateway from a Python Client

- ✓ Defines the list of enabled servers from the gateway's command line, with `--server`
- ✓ Uses the latest http streaming transport.
- ✓ Uses the online Docker MCP Catalog hosted on <http://desktop.docker.com/mcp/catalog/v2/catalog.yaml>.



```
services:
  client:
    build: .
    environment:
      - MCP_HOST=http://gateway:9011/mcp
    depends_on:
      - gateway

gateway:
  image: docker/mcp-gateway
  command:
    - --transport=streaming
    - --servers=duckduckgo
    - --verbose=false
    - --port=9011
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
```

Agentic applications need three things



Models 



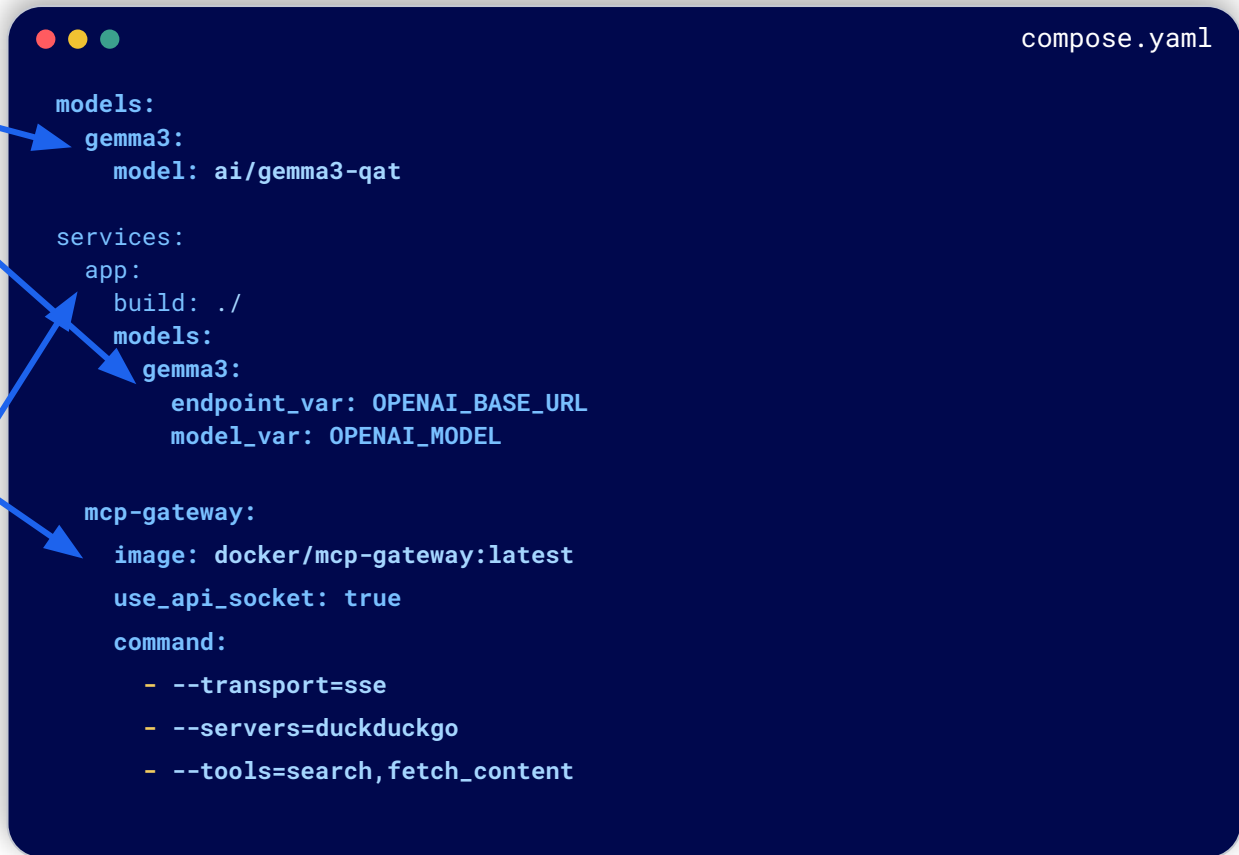
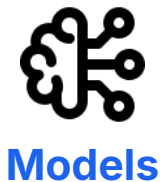
Tools 



Code



Bringing it all together





Coding Agents

AI Agents are here. And they're doing real work.

“

84%

of developers now use
or plan to use AI tools

Stack Overflow Developer Survey 2025

“

42%

of all code written
today
is AI-assisted

Sonar State of Code 2025

“

74%

of executives deploying
AI Agents see ROI in
the first year

Google Cloud ROI of AI Report 2025



Agents are the biggest productivity shift in decades

ENGINEERING

**Read whole codebases.
Ship pull requests.**

Agents read context, write code, and open PRs without an engineer in the loop.

MARKETING

**Pull CRM data.
Launch campaigns**

Agents move from research to creative to send, end to end.

FINANCE

**Reconcile reports.
Query systems live.**

Agents close the loop between ledger, dashboard, and decision.



←  r/ClaudeAI • 1mo ago
Substantial_Word4652

An AI agent deleted 25,000 documents from the wrong database. One second of distraction. Real case.

Bug

I'm going to be completely honest because I think this can happen to anyone who rather you learn from my scare than live it yourself.

The context

I was getting a project ready for production. The database was full of mock data keeping certain specific data so I wouldn't have to regenerate everything. The .env.local with the right credentials, scripts perfectly referenced, docs in /docs the whole structure.

←  r/AgentsOfAI • 1mo ago
ailovershoyab

I just watched my AI Agent delete 400 emails because it thought they were 'clutter.' We are officially in the Wild West of 2026.

Discussion

I finally caved and set up OpenClaw (the viral agentic tool everyone's talking about this month) to help "triage" my inbox. I was like, "I'll use this to clean up my inbox and delete anything that isn't a priority."

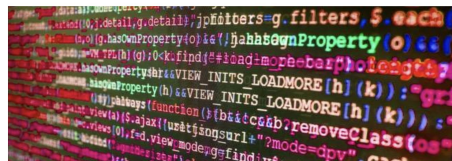
New Vulnerability in GitHub Copilot and Cursor: How Hackers Can Weaponize Code Agents



By Ziv Karliner
March 18, 2025

Researchers warn of prompt injection vulnerability in GitHub MCP with no obvious fix

By Tim Anderson • May 27, 2025



←  r/AI_Agents • 2mo ago
Informal_Tangerine51

Agent deleted production data because no policy layer said 'no' - what's your governance strategy?

ment intake agent deleted production data last week. Not because of a bug. Because it old and should be cleaned up. It was technically allowed to do this. It shouldn't have been.

. We have an agent that processes incoming documents, extracts fields, and writes on database. Works great most of the time. Processes hundreds of docs per day.

tered a batch of documents that looked similar to records already in the database. The look like duplicates of old data. The old records are from 6 months ago. They're probably am up before adding the new ones."

records API. The API executed. 1,200 customer records gone.

JUL 9, 2025 3:28 PM

McDonald's AI Hiring Bot Exposed Millions of Applicants' Data to Hackers Who Tried the Password '123456'

Basic security flaws left the personal info of tens of millions of McDonald's job-seekers vulnerable on the "McHire" site built by AI software firm Paradox.ai.



Speed or safety? This is the tension.

The dilemma every Engineering Leader is facing.

Speed

Need autonomy but can't let agents run wild

- Agents delete months of work.
- Secrets get exposed.
- Critical systems get damaged.
- One bad action, real consequences.

Safety

Lock everything down

- Approve every file read.
- Approve every tool call.
- Approve every action.
- Safe, but defeats the purpose.



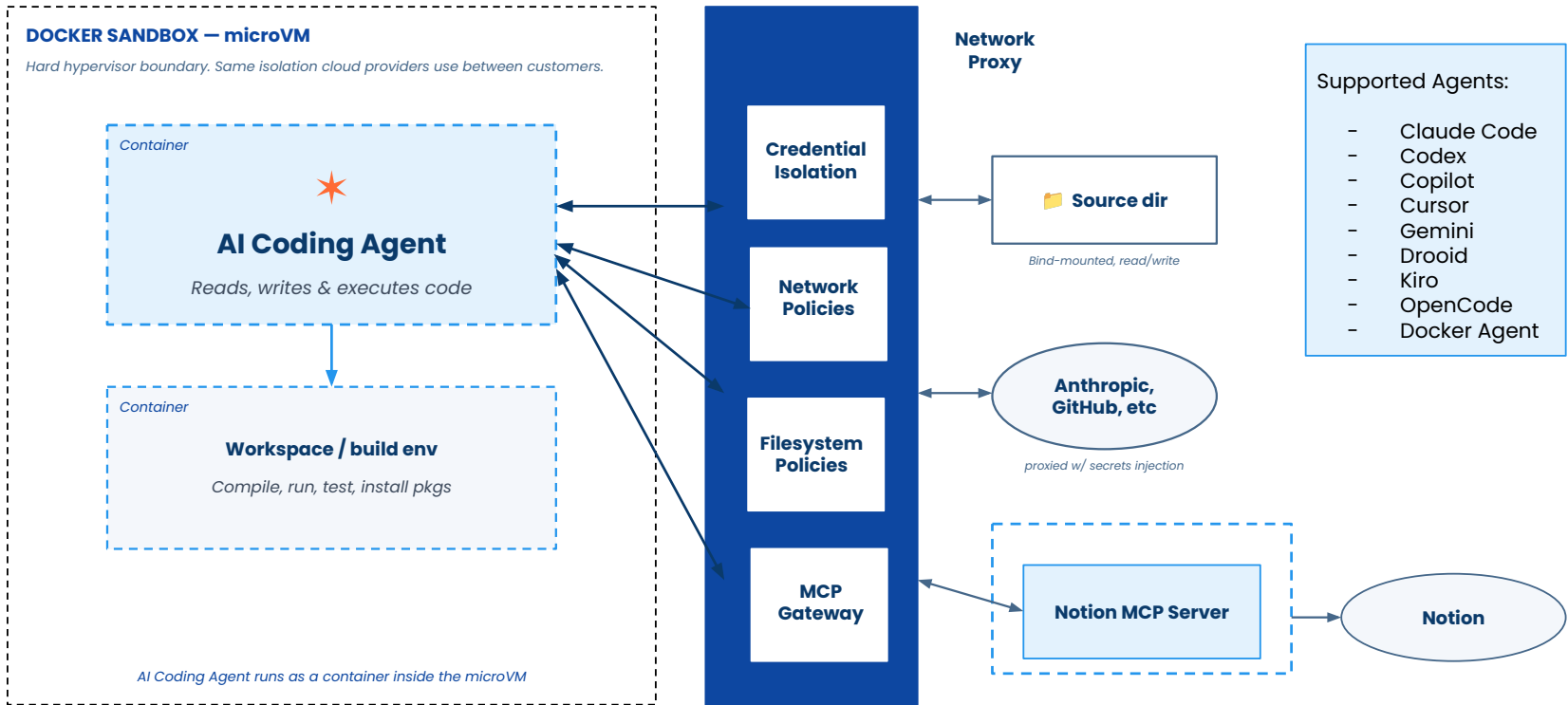
**Why not
get both?**





Introducing Docker AI Governance

Docker AI Governance



Coding agent runs in a microVM because it executes code. Every external action goes through Docker AI Governance.





Sandboxing the agent

Experimental

Sandboxes

Run agents in isolation. Rather than on your bare machine.

- 1 **Agent or claw runs in an isolated sandbox** that mirrors your workspace
- 2 **You define what access** it gets to your filesystem, network, internal resources, and tools
- 3 Your real files, data, secrets **stay safe even if the agent or claw goes off rails**



```
$ sbx run claude
Starting claude agent in sandbox 'claude-ai-project'...
:: Mounting workspace: ~/projects/ai-project
:: Network policy: deny all, allow 42 hostnames

Claude Code v2.1.72

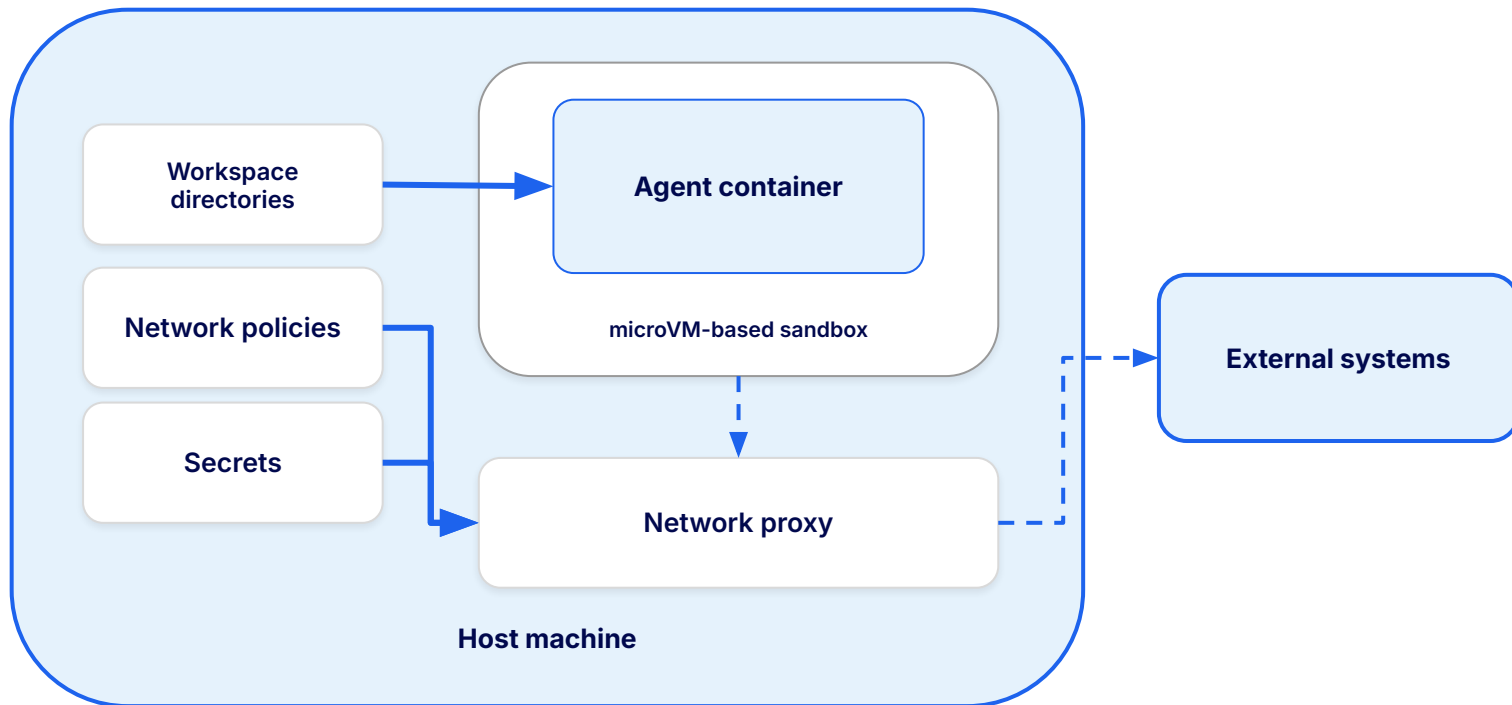
Welcome back!

Sonnet 4.6 · API Usage Billing
/Users/moby/projects/ai-project

> Do something incredible, but with proper agent guardrails!
```

Tips for getting started
Ask Claude to help you
Recent activity
No recent activity

Sandbox architecture



Install. Sign up. Go. **No Docker Desktop required.**



```
brew install docker/tap/sbx
```



```
winget install Docker.sbx
```

Sandboxes

Docker

```
$ sbx run claude
Starting claude agent in sandbox 'claude-ai-project'...
:: Mounting workspace: ~/projects/ai-project
:: Network policy: deny all, allow 42 hostnames
```

Claude Code v2.1.72

Welcome back!



Sonnet 4.6 · API Usage Billing
/Users/moby/projects/ai-project

Tips for getting started
Ask Claude to create a new app or clone a repository

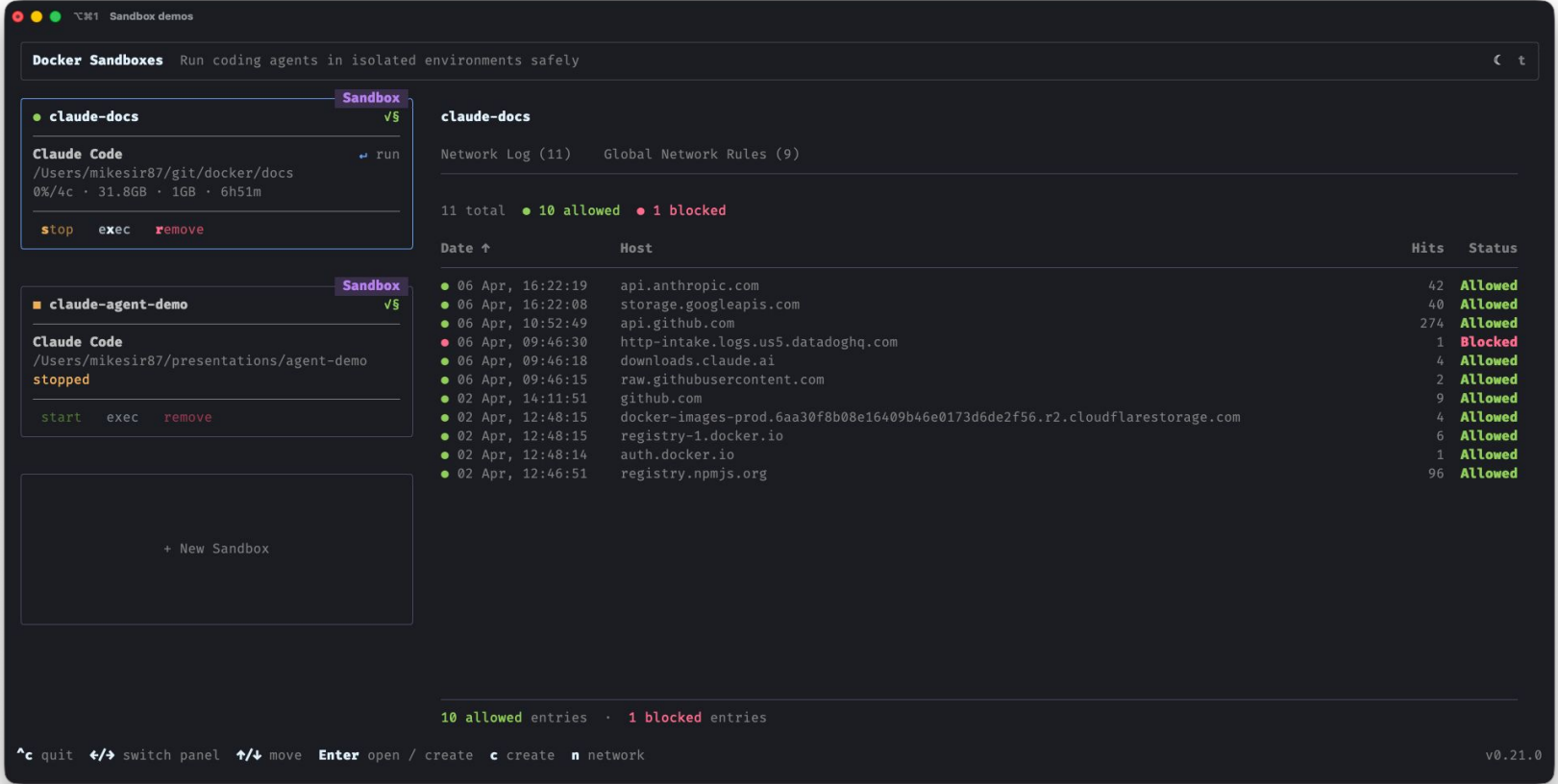
Recent activity
No recent activity

> Do something incredible, but with proper agent guardrails!

Key CLI Commands - Cheat sheet

<code>sbx run claude</code>	Start (or reconnect to) a sandbox for the current directory
<code>sbx ls</code>	List all sandboxes with status, uptime, and port mappings
<code>sbx stop <name></code>	Pause the sandbox VM (state is preserved)
<code>sbx rm <name></code>	Delete the sandbox and everything inside it
<code>sbx exec -it <name> bash</code>	Open an interactive shell inside a running sandbox container
<code>sbx ports <name> --publish 8080:3000</code>	Forward host port 8080 → sandbox port 3000
<code>sbx policy allow network '*.npmjs.org'</code>	Allow a domain through the network policy
<code>sbx policy log</code>	Show blocked and allowed outbound connections
<code>sbx secret set -g anthropic</code>	Store an API key in your OS keychain
<code>sbx</code>	Open the interactive TUI dashboard

The Sandbox TUI



Managing credentials

- **Always** prefer stored secrets over env vars - keychain encrypts at rest
- **Never** set API keys manually inside the sandbox - they'll be readable by the agent

CLI

```
# Store key in OS keychain - available to all sandboxes  
sbx secret set -g anthropic
```

```
# Scoped to a specific sandbox  
sbx secret set my-sandbox openai
```

```
# Pass GitHub token so the agent can use gh CLI  
gh auth token | sbx secret set -g github
```

```
# List and remove  
sbx secret ls  
sbx secret rm -g github
```

Supported services:

- anthropic
- openai
- github
- google
- groq
- mistral
- xai
- aws
- nebius

Demos



Basic walkthrough



Credential isolation



Network isolation



Filesystem isolation



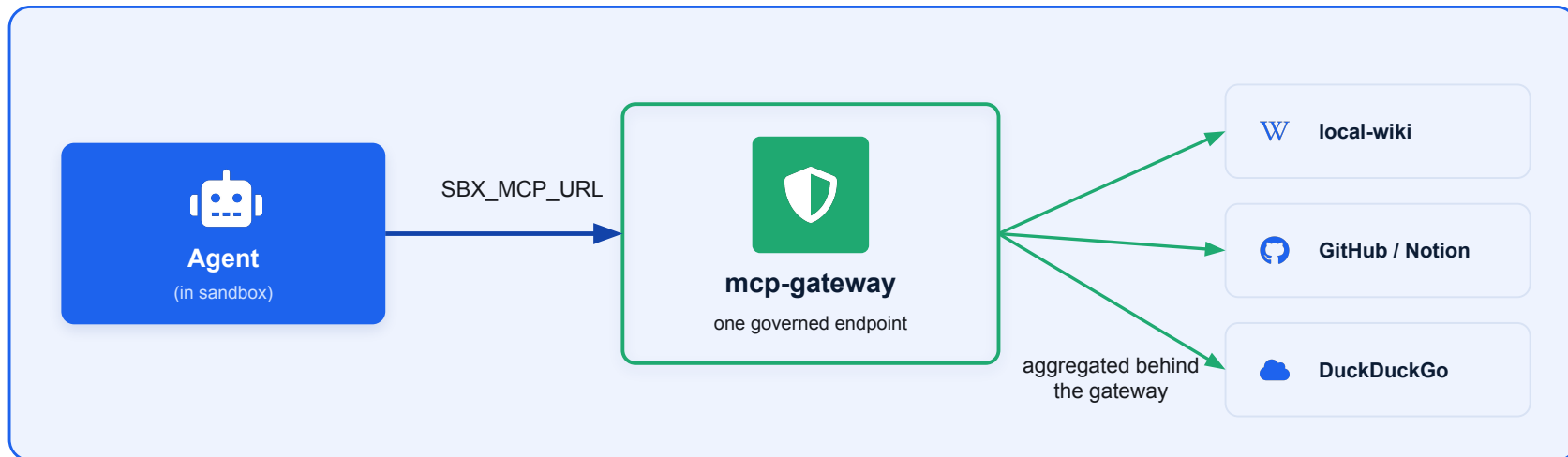


Protecting MCP

Tools are how agents act on the world. Govern which servers exist and which tools an agent may actually call at one gateway.

LETTING AI AGENT INSIDE SANDBOX USE MCP SERVERS SAFELY

The agent talks to one gateway, never to servers directly



Every tool call flows through one chokepoint. Tools are namespaced `mcp__mcp-gateway__<tool>` that's where policy + audit apply.

Point it at a real gateway, nothing else works

The `sbx mcp` subtree stays hidden until `SBX_MCP_URL` is set. Only two values carry the full governed flow:



Local gateway

```
http://localhost:8811
```

Compose or Desktop MCP Toolkit · you run it,
you control what's registered



Best for learning the mechanics



Hosted control plane

```
https://gateway.docker.com
```

MCP Gateway Enterprise · org policy governs
what's invocable · central audit

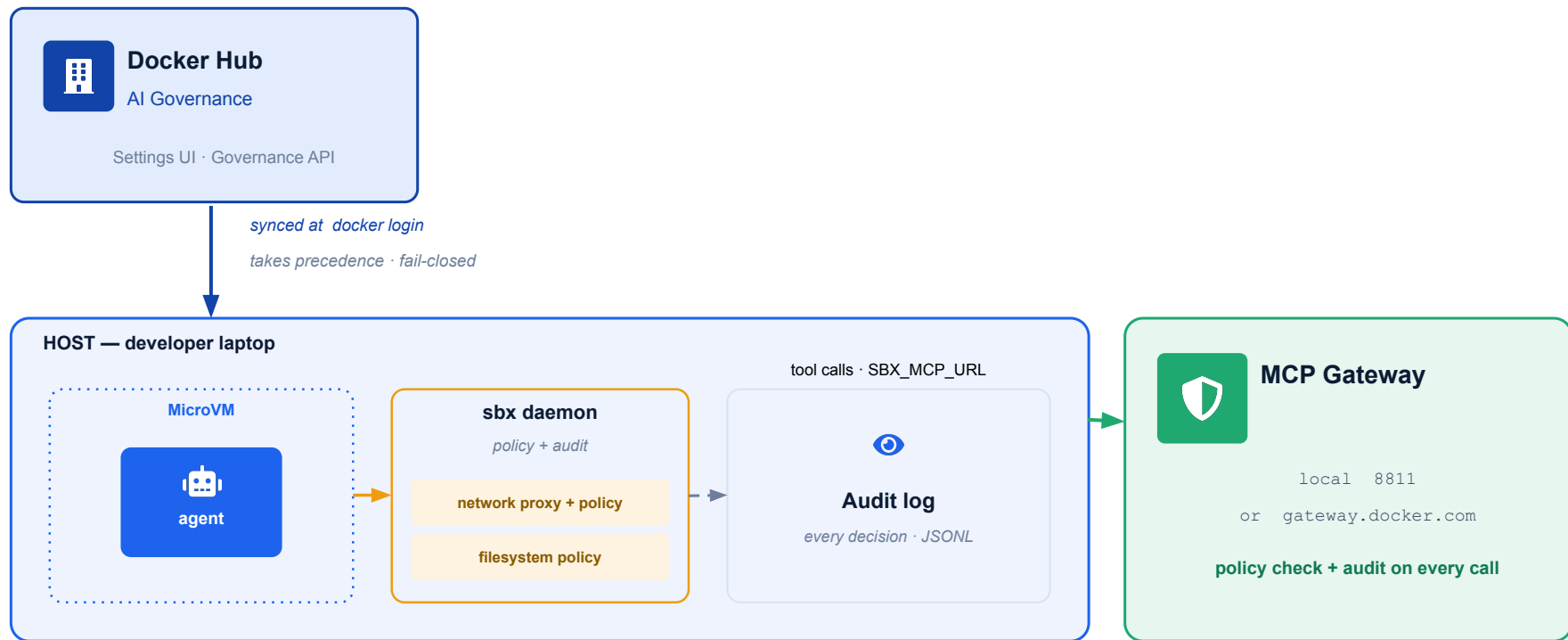


The real governance story



Fail-closed by design. No policy loaded $\Rightarrow$ deny-all. Policy is authored in Docker Hub, fetched at `docker login`, and developers can't override it.

Author once in Docker Hub, enforce around the sandbox and at the gateway



The server lifecycle - five commands

1

add

Register a server

```
sbx mcp add local-wiki \  
  --command docker \  
  --args "run,-i,--rm,mcp/wikipedia-mcp"
```

list, stdio, remote-OAuth or docker.io image

2

ls / inspect

See the record

```
sbx mcp ls  
sbx mcp inspect local-wiki
```

registration only — doesn't start the server

3

--static-mcp

Attach to a sandbox

```
sbx run claude --static-mcp local-wiki  
# or into a live one:  
sbx mcp load local-wiki
```

the flag is --static-mcp, NOT --mcp

4

/mcp

Verify inside the agent

```
/mcp  
→ mcp-gateway · connected · 24 tools
```

one aggregated gateway, not your servers

Default-deny allow-list over (server, tool), authored in Cedar

Cedar is an open-source policy language and authorization engine created by AWS. Can principal X do action Y on resource Z with context C.

CEDAR POLICY · [app.docker.com](#) → AI governance

```
permit(  
  principal,  
  action == MCP::Action::"invoke",  
  resource is MCP::Tool  
)  
when {  
  resource.server == "github-official" &&  
  resource.name == "get_me"  
};
```

What this permits



Exactly one tool: `get_me` on `github-official`



Every other tool and every other server - blocked



Evaluated at the gateway on every `invoke`



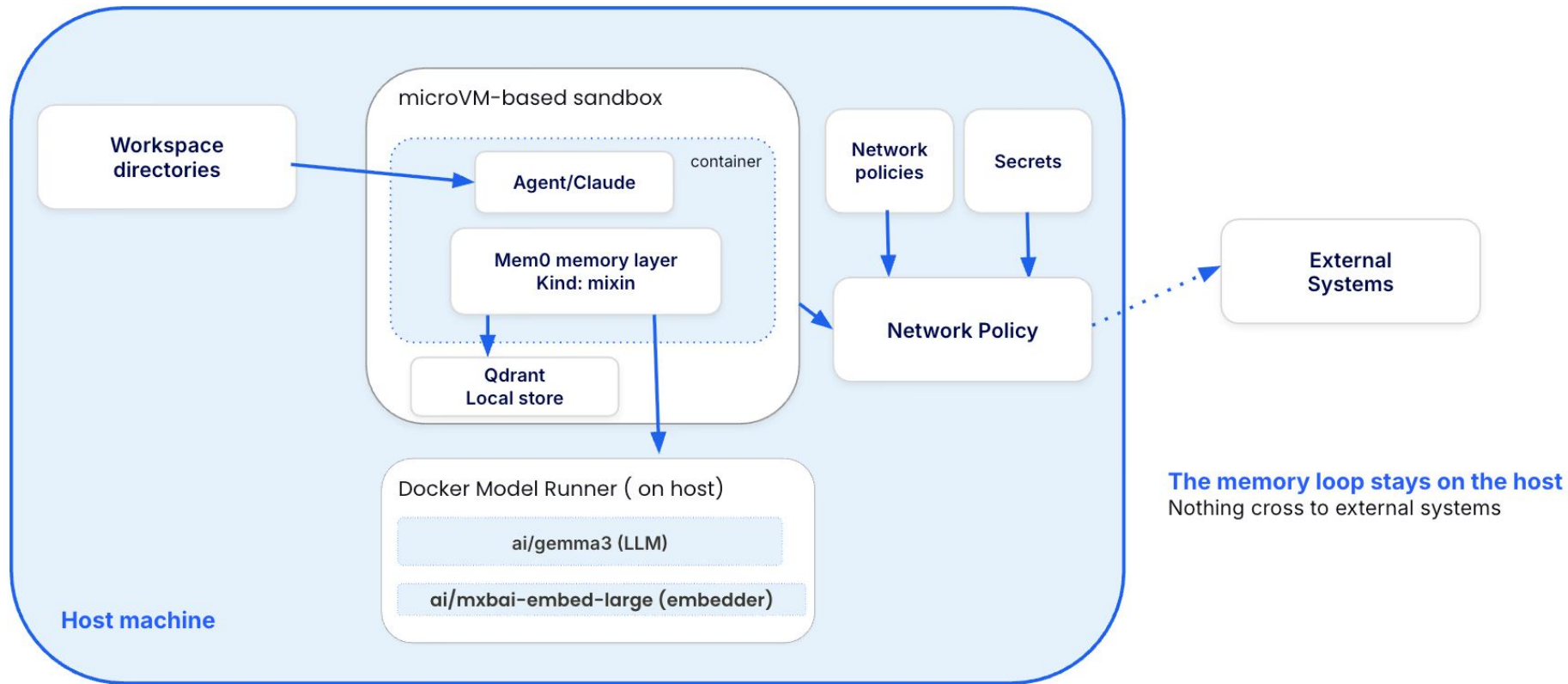
Same engine as network + filesystem policy - one surface, no bypass

Author once, sync everywhere. A developer can `sbx mcp add` any server, if org policy doesn't permit its tools, the calls are denied and audited.

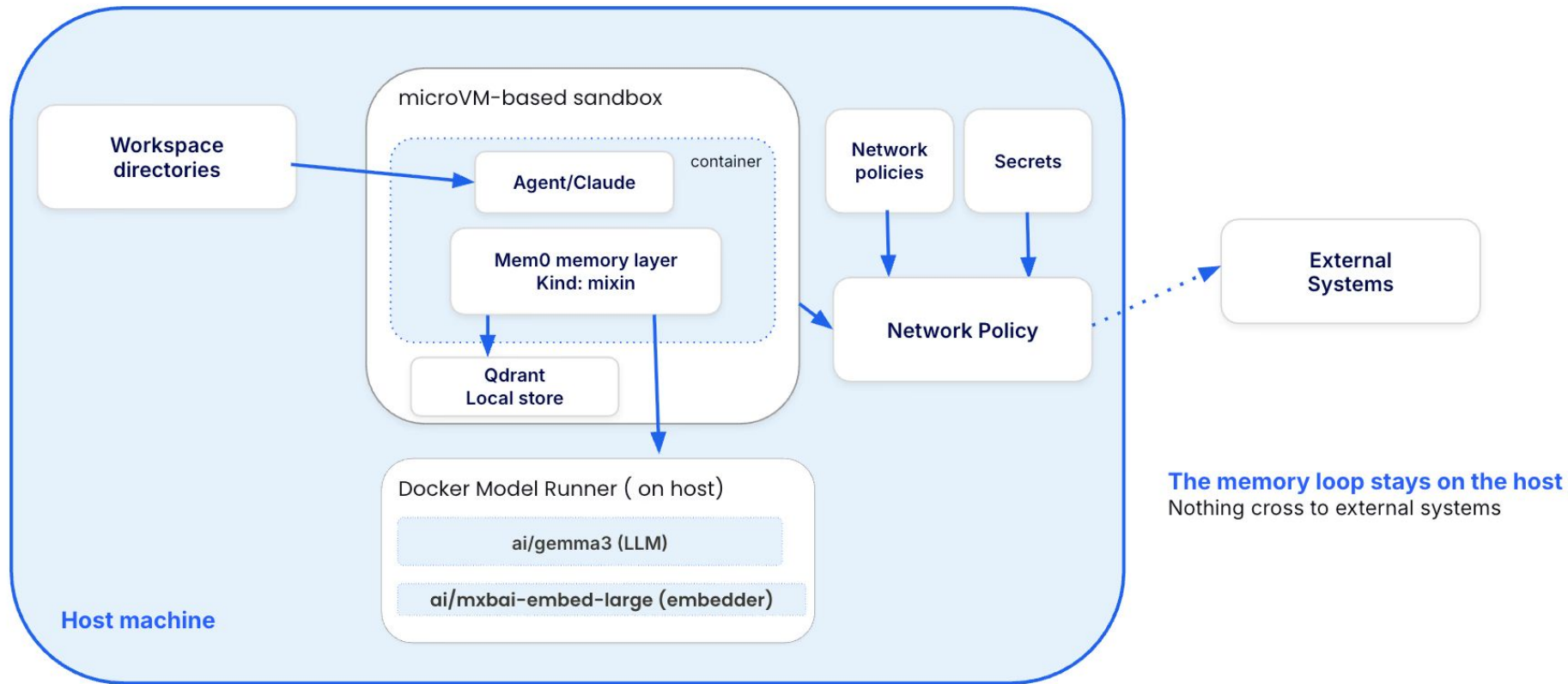


Extending Sandbox with kits

The base sbx microVM gives you the isolation boundary. A kit layers tooling, service wiring, credentials plumbing and any policy allowances.



<https://github.com/ajeetrainai/sbx-kits-mem0>



<https://github.com/ajeetrainai/sbx-kits-mem0>

A layered approach to AI governance

Every layer adds an enforceable control and the policies travel with the workload, whether it runs on a laptop or in the cloud.

DOCKER BUSINESS + AI GOVERNANCE POLICIES

Gordon

In-product governance guidance, inside Docker Desktop

Agentic Compose & Docker Agent

Declarative multi-agent orchestration — secure golden templates

Docker Model Runner

Local, air-gapped LLM execution for your workloads

MCP Toolkit & Gateway

Access only to MCP servers the organization authorizes

Docker Sandboxes

Isolated, portable runtime — policies travel with the sandbox

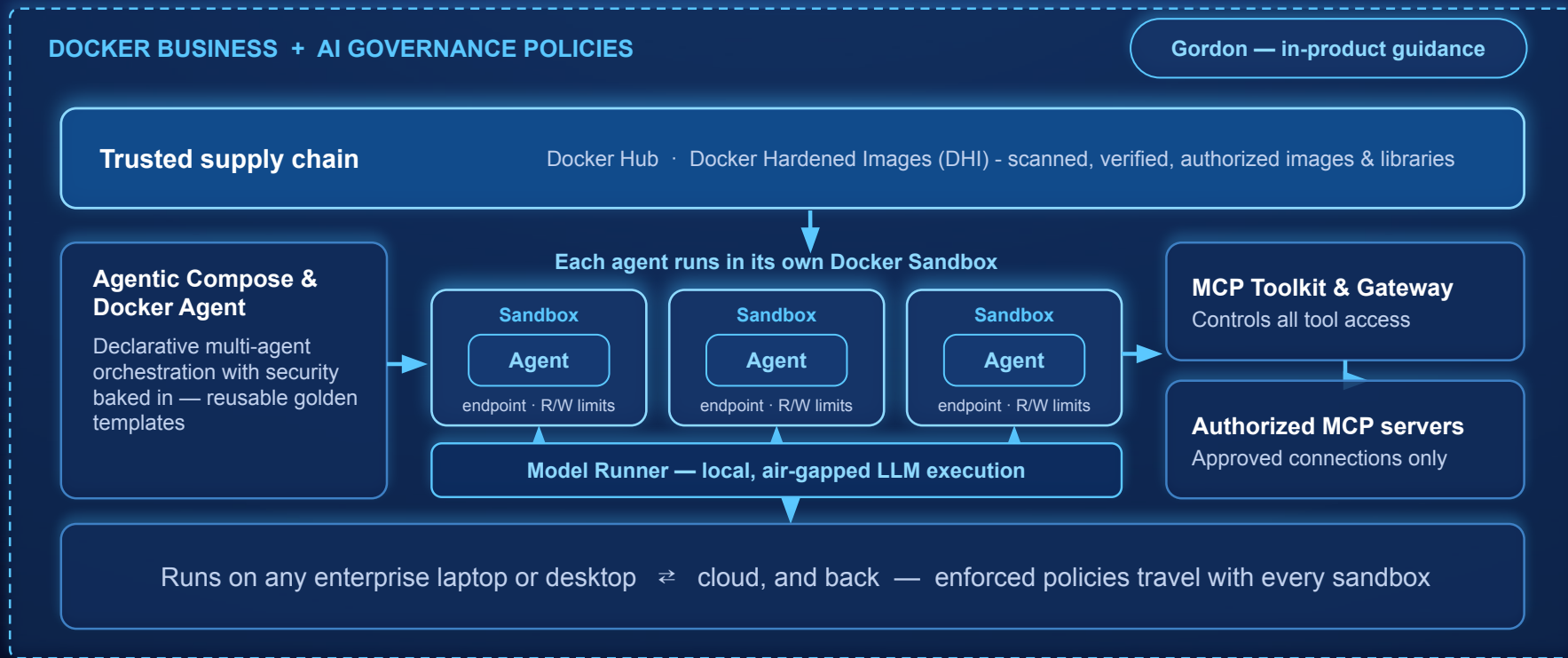
Docker Hardened Images (DHI)

The trusted foundation: scanned, verified, authorized



The Docker AI governance ecosystem

All six layers working together as one governed system - on the laptop or in the cloud.



References

- <https://dockerworkshop.com/>
- <https://www.docker.com/products/model-runner/>
- <https://www.docker.com/products/hardened-images/>
- <https://www.docker.com/products/docker-scout/>
- <https://github.com/dockersamples/catalog-service-node>





Thank You.